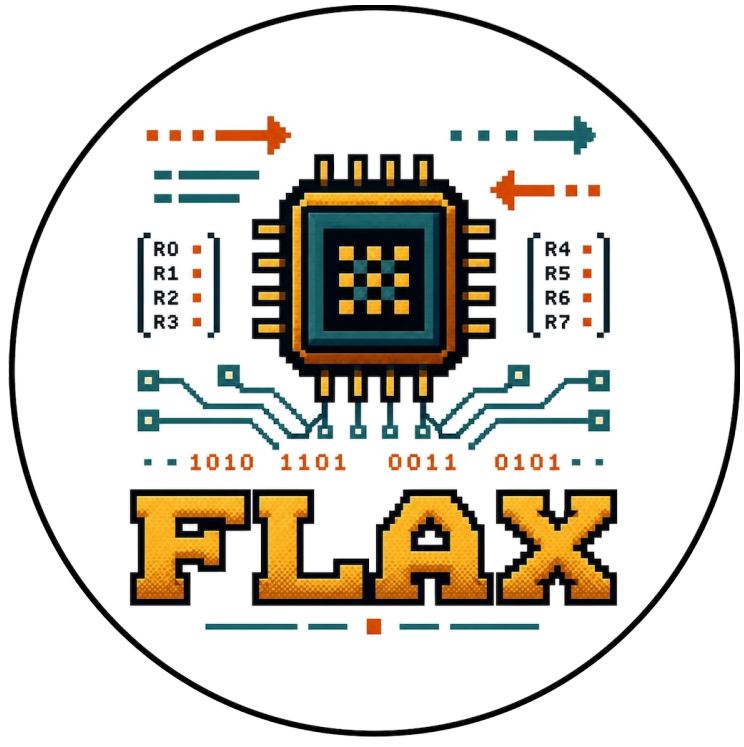
FLAX

SYSTEM MANUAL

BETA 1.5



Copyright (c) 2026 The Flax Project

This manual reflects the feature set included in the Beta 1.5 build.

Any documented feature may change in functionality or appearance, or may be removed entirely before the final release.

If you encounter an issue with this manual or the Flax system, please report it to the development team through the GitHub issue tracker.

https://github.com/ryncarmichael/Flax

Last Updated: 14/10/2026

Introduction 1
Design 2
License 3
Requirements 6
Recommended System Requirements 6
Limitations 7
Performance 8
Security Considerations 9
Installation 10
Downloading 10
Copying to Preferred Directory 10
Updating Path 11
Testing Installation 11
Recommended Terminal Configuration 12
Font Choice 12
Execution And Use 13
Compiling 13
Command Line Switches 14
Inline Switches 14
Encrypting Your Flax Code. 15
Running Your Program 16
Command Line Switches 16
Interactive Mode 16
Shell Scripts 17
Editing Code 18
Editor Choice 18
Integration Scripts 18
Language Modules 18
Style Guide 19
Your First Program 20
Hello Flax World! 20
Reference 23
Case Matters 23
Code Profiling 23

Comments 24
Compiler Flags And Switches....25
Constants 26
Generic Constants....26
Instruction Specific Constants (words)....26
Error Messages And Warnings 28
Compile Errors....28
Compile Profiling Errors 29
Compile Warnings....30
Runtime Errors....30
General System Errors:....32
File IO....33
Frequently Asked Questions....35
What does the name Flax mean? 35
What operating systems does Flax support?....35
Could Flax be used to write a virus? 35
What can Flax be used for?....35
How much does Flax cost?....35
Flags And Key Values Pairs....36
Flags....36
Key Value Pairs....37
Instructions 38
Instruction Set 38
Parameter Types....39
Syntax, Patterns and Variations....39
IRQ System....51
Subroutines 53
Registers 55
Terminal Control 56
Timers 59
Variables....60
Declarations 61
Using Offsets....63
Virtual Ports....64
Yield Phasing....65
System Yield....65

Auto Pause 65
IRQ activation 66
Change Log 67
Flax Change Log 67

INTRODUCTION

Welcome to Flax, a general-purpose, high-speed assembly language compiler and runtime, which has often been used for teaching low-level programming concepts.

Even though Flax is the latest incarnation of what was and has been developed in-house over many years and is still fit for its original purpose, Flax's new design makes it ideal for use outside of the education space as well.

From traditional or personal education, entertainment, high-speed scripting, and even cluster-based applications. The list of possibilities is endless.

(1) Although Flax code is assembly-like in nature, it is not assembly language in the strictest sense, as it is not tied to a specific hardware platform or CPU vendor's instruction set.

(2) Flax is not, and is never intended to become, a game engine. A GUI-based version may be introduced in the future, but it will not include the complexity or feature set of a conventional game engine.

(3) The Flax Language Project is entirely separate from the Flax Game Engine. It does not reproduce, compete with, or incorporate features from that engine. The Flax Language Project is the continuation of a language developed for educational purposes over many years.

DESIGN

Flax is not just an update to an earlier version with reworked legacy code and a fancy new front end. It has been completely rewritten from scratch, utilising the latest in high-speed languages and compilers.

To say that Flax is fast would be an understatement. Prior versions saw instructions per second counts at most of about one hundred million instructions per second at best. Flax regularly breaks the six hundred million barrier, making it suitable for more than just learning or as a novel toy to play with.

To achieve these results, we tested many modern, performant languages and compilers by running the same performance testing code we've run ever since the very first version, before finally settling on the one language which provided both the best performance and just *clicked* with the development team, allowing them to get up to speed very quickly.

Flax in itself is not multi-threaded, which might seem odd in this day and age. From the very first version, we've tried to emulate the design philosophy of early microprocessors, but that doesn't mean you can't appear to do more than one thing at a time. With an IRQ *(Interrupt)* system, unlimited timers, and a full suite of branching instructions, you can easily create as many sudo threads as you'd like, without the dangers and complexity of making everything thread-safe.

The biggest problem we've had during development was discovering how to slow it down.

Although Flax appears assembly-like in nature, its default instruction set excludes such things as binary operators or carry flags you'd encounter in true assembly coding. Flax was originally designed to fill a niche role, one in education, where its reduced instruction set would promote creative thinking and problem-solving skills while still teaching low-level programming concepts.

Even though education usage is still high on our agenda, there is no reason that Flax couldn't be used beyond the confines of education as a high-speed scripting language, entertainment, or for cluster computing tasks. Personal education and even just for the fun of doing something different are also driving factors.

LICENSE

Flax Language, Compiler and Runtime Non-Commercial License

Version 1.0 — Copyright (c) 2026 The Flax Project

1. Grant of License

Subject to these terms, the copyright holder grants you a non-exclusive, worldwide, royalty-free license to download, install, and use Flax for personal, non-commercial, or educational purposes.

This license is granted to an individual user. Educational use includes use by students, educators, and educational institutions for teaching, learning, coursework, or academic research, provided that the use is not primarily intended to generate revenue.

2. Commercial Use

Commercial use is not permitted under this license.

Commercial use includes:

- use by a business, company, or other commercial organisation;
- use by employees or contractors in connection with their work;
- use in providing paid services or consulting;
- use in a product or service offered for payment;
- use to support revenue-generating activities; and
- use in an internal business workflow.

Commercial use requires a separate written commercial license from The Flax Project before such use begins. Commercial licenses may be priced per authorised user or seat. Pricing, subscription terms, lifetime terms, support, and update rights will be specified in the applicable commercial license agreement.

3. Evaluation Use

A commercial organisation may evaluate Flax for up to 30 days solely to determine whether to purchase a commercial license. Evaluation use may not be used to perform production work or provide paid services.

4. Restrictions

Except as expressly permitted by this license or a separate written agreement, you may not:

- use Flax for commercial purposes;
- sell, rent, lease, sublicense, or redistribute Flax;
- provide Flax as a hosted or managed service;
- remove copyright, license, or attribution notices; or
- use Flax to create or distribute a competing product.

5. Distribution

You may download the official Flax installer from an authorised distribution channel and make an unchanged copy solely for your own personal backup.

You may not mirror, repackage, modify, sell, rent, sublicense, or redistribute the installer or any component of Flax without prior written permission.

You may share links to the official Flax download page, but you may not host or redistribute copies of the Flax installer.

6. Ownership

Flax is licensed, not sold. The Flax Project and its licensors retain all rights, title, and interest in Flax except for the rights expressly granted by this license.

7. Termination

This license terminates automatically if you breach its terms. Upon termination, you must stop using Flax and delete all copies in your possession or control.

8. Disclaimer of Warranty

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, FLAX IS PROVIDED "AS IS" AND "AS AVAILABLE," WITHOUT WARRANTIES OF ANY KIND, WHETHER EXPRESS, IMPLIED, OR STATUTORY, INCLUDING WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, AND NON-INFRINGEMENT.

9. Limitation of Liability

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE FLAX PROJECT AND ITS CONTRIBUTORS SHALL NOT BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXEMPLARY, OR punitive DAMAGES, OR FOR ANY LOSS OF DATA, PROFITS, REVENUE, OR BUSINESS INTERRUPTION, ARISING OUT OF OR RELATING TO FLAX OR THIS LICENSE.

REQUIREMENTS

The Flax runtime and compiler have the following system requirements and should be compatible with most relatively modern macOS systems.

Recommended System Requirements

Operating System	Operating System Version	Hardware Specifications	Development Status
MacOS	Current macOS release or the previous major release.	Apple silicon processor or x86_64 processor for Intel Macs. 2–4GB of available memory.	Beta 1.5

The only additional requirement is a terminal emulator capable of interpreting ANSI/ASCII escape sequences, including colour codes; thankfully, macOS's default terminal has native support for all without any modifications.

Flax has always been a macOS only application, though if the necessary cross-platform compilation tooling were ever released, we *might* consider adding Linux support. Windows support would never be possible as Flax requires a terminal capable of interpreting UNIX-style commands, which Windows does not support.

**The default macOS terminal using ZSH as the shell works fully without any modification whatsoever. The default terminal setting of xterm-256colour is ideal and is the recommended configuration.*

LIMITATIONS

The initial version of Flax is limited to being a macOS terminal-only application, but that doesn't mean you can't achieve some pretty amazing results.

With true colour ANSI/ASCII support and complete control over cursor column and row positioning, anything is possible.

A version of Flax with a GUI and GPU-accelerated features is planned once the engine is complete and the overall concept proven, though it will not have any of the complexities of a true game engine.

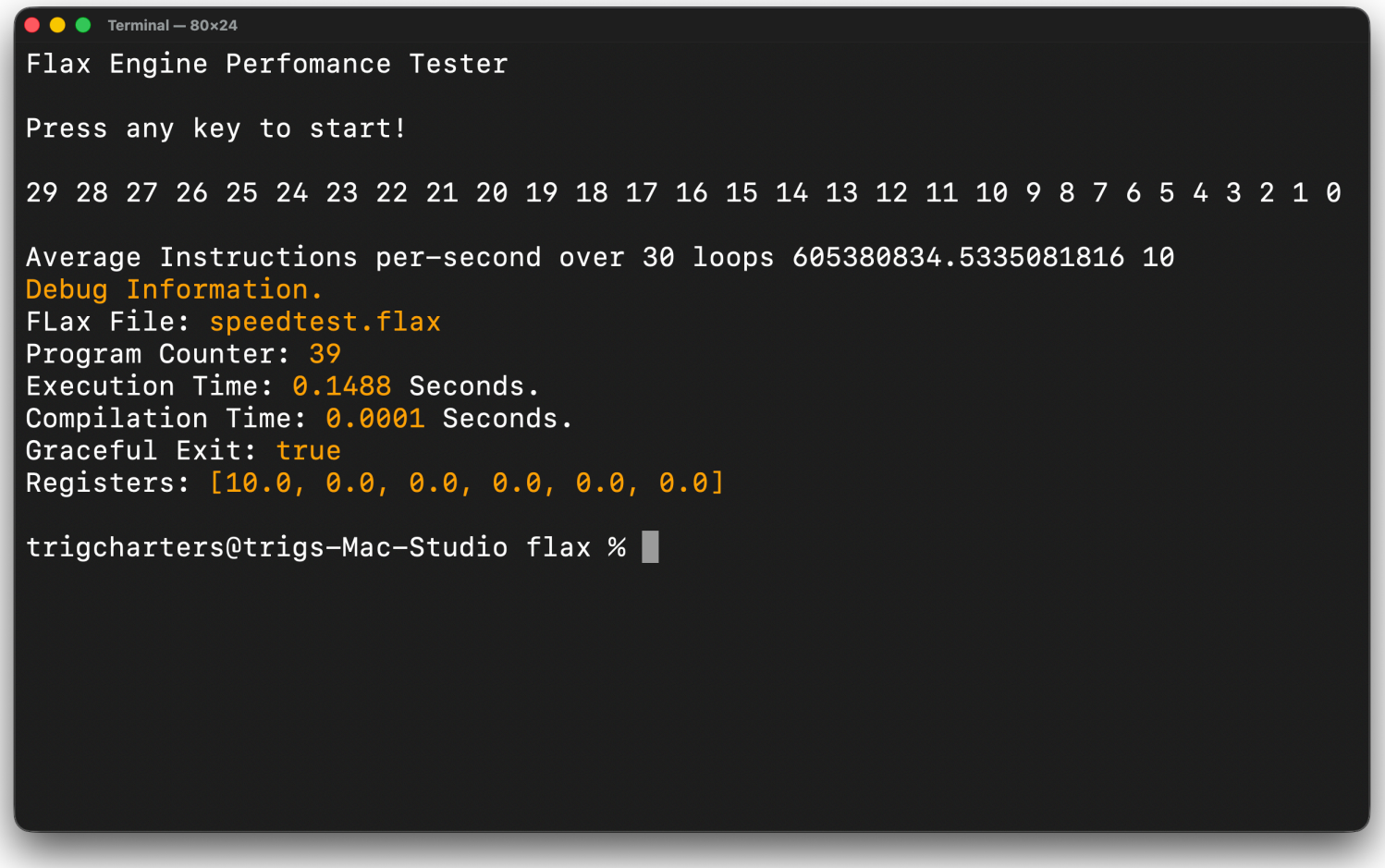
PERFORMANCE

I accept that performance benchmarks are usually entirely useless as they aren't performing any real-world task and are just fluff to inflate the developers' ego.

In this case, however, they have a use, if only for us. As we've run exactly the same code on every prior version and have those performance numbers, they show a clear improvement over time. The prior versions ran at **3**, **13**, **80**, and **145** *(MIPS)*, and the latest **600**+.

This doesn't mean that any of the prior versions were bad; they were perfect for their intended use case, and I doubt you could even tell the difference between the version running at 3 and the latest version in everyday use.

All this benchmark really shows is there is a lot more performance headroom available in this version than the previous.



SECURITY CONSIDERATIONS

Flax has been designed to adhere to all macOS rules and preferences. It will not try to access areas outside of those which you have given it access to, and such areas should include no more than the Documents folder in which you've chosen to store your Flax projects. It has no need, and never will, require access to the network or Internet.

On this point, because of such security restrictions, Flax does not have any capability to check for updates or service patches. That task will fall upon you as the user, though in most cases, unless there is a specific new feature or a required bug fix, there is no requirement to upgrade, and we as the developer will attempt to provide support for older versions for as long as applicable.

We can also include you in an email list, should you desire, to inform you of bug fixes, updates, or new versions when they are released.



INSTALLATION

The entire Flax system is less than 500kb in size fully installed, not megabytes, kilobytes, which is not the normal case in this day and age. In fact, this system's manual, which you can download separately, is far larger than the whole of the Flax System combined, and there are no additional libraries or runtime files required except what is already included within the installation package.

The package itself only includes three files: the '**runflax**' shell script wrapper for the main executable, the '**flax**' executable itself, and lastly a single library file '**flax.elib**'.

Given such a small footprint, installation only takes seconds.

DOWNLOADING

Flax is available from our '**GitHub**' repository. The latest version will be listed on the front page, but historical versions will always be available within the package history.

Flax will be made available in a compressed '**ZIP**' file or other compressed formats should the need require.

GitHub Address: https://github.com/ryncarmichael/Flax

COPYING TO PREFERRED DIRECTORY

As version 1.0 of Flax is a console application, it is best installed somewhere in the console's search path.

We recommend placing the Flax executable and support files in:

'**usr/local/bin/flax**'

UPDATING PATH

Wherever you place the Flax Package, you will need to update your terminal environment with the path to your chosen installation directory. For the recommended directory listed above, add the following to your terminal login file.

On macOS using '**zsh**' as the shell, the following would be required in the '**.zshrc**' file in your home directory.

export PATH="/usr/local/bin/flax:$PATH"

Or use the following to make the update even easier.

echo 'export PATH="/path/to/Flax/folder:$PATH"' >> ~/.zshrc

TESTING INSTALLATION

After the above is done to test the installation, restart the terminal then just type '**runflax**' to test the installation. If a '**file not found**' message is displayed, the most probable cause is that you have either not restarted the terminal completely or the path to your installation directory is incorrect.

Note: For early access and beta releases, the package will not be code-signed as this is a process best suited for official releases. On macOS, in this case, you may need to approve the application by first selecting '**Open Anyway**'. Then, in '**System Preferences / Privacy and Security / Security',** click on the button to allow the Flax executable to run. Running Flax for the first time, you may have to repeat the above steps for both the '**runflax**' process and the '**flax**' process, which is the native executable.

Another possible solution to the above notification on macOS is to run the following command in the terminal, which does everything above but all in one go. I prefer to use the '**System Preferences**' approach as and it's more granular and shows exactly what processes you're approving, though if you've installed Flax into its own folder then you're covered either way.

xattr -d com.apple.quarantine <path_to_flax_executable>

RECOMMENDED TERMINAL CONFIGURATION

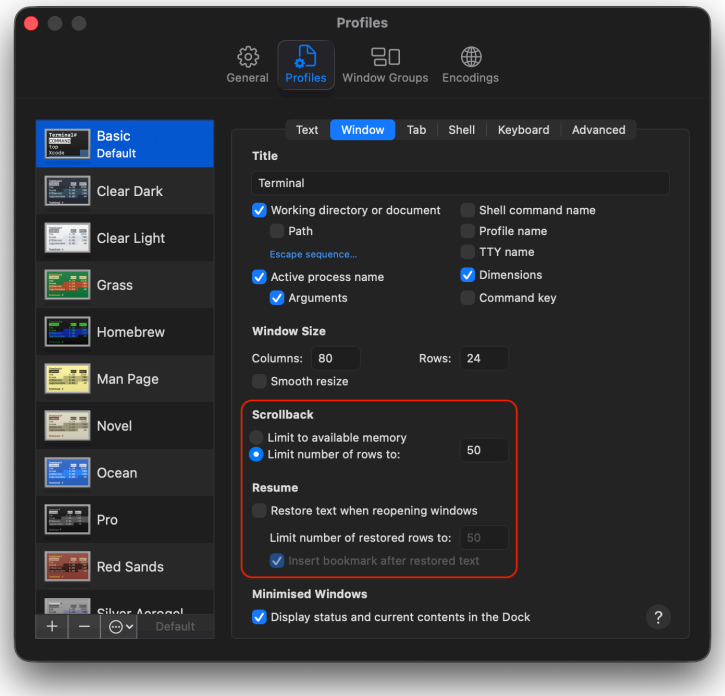
Most terminals have a configuration option for the amount of memory or number of scroll-back lines being stored, both when the terminal is running and what is restored after exiting and reloading.

Many terminals set this value to '**available memory**' or a very high number, which, when the terminal is running, may appear as a memory leak as the amount of memory consumed by the terminal process will continue to grow as the amount of scroll-back history is retained.

We recommend adjusting this value to a more conservative setting. A value of 100 or less is appropriate as it is unlikely most users would ever need to scroll back further than that.

Our recommended configuration on macOS for running Flax is shown below; you can also create profiles for other situations if you don't want to make this change global.

One positive side effect of making this change is a noticeable performance increase in both the terminal and Flax.



Font Choice

When configuring your terminal, you should also take into consideration the font being used. It's best to use a mono-spaced, fixed-width font, but also one with regular, bold, italic, and bold italic styles. We recommend '**Menlo',** as it has all the styles along with a huge number of specialised UTF characters.

EXECUTION AND USE

Flax, when configured as default, is extremely simple to use. The Flax executable contains both the compiler and runtime, so there is no need to remember complex build processes or a myriad of compile-build applications or cryptic configuration switches.

Flax utilises a custom-designed pattern matching compiler with a specialised constants mapping system to enforce strict instruction-to-constant syntax. During the compile process, the Flax code is also profiled to identify generalised issues that are not easily detected with simple pattern matching.

The compiler also checks for the existence of a '**_start:**' label for setting the program counter's position on execution. There are IRQ system checks as well as a differential examination of subroutine jumps to return statements, which might indicate missing or unbalanced subroutine jumps and '**rts**' instructions.

Finally, there is also the check for a valid end point in the program, which, if missing, would cause the program to exceed the bounds of memory with the potential of causing a system error.

If you are unsure about the exact use of Flax, you can enter the '**runflax**' command in the terminal with the '**-h**' switch to display the inbuilt help and a list of all other switches available.

Note: command line switches are not case-sensitive, though the Flax executable and source file names are.

```
runflax -h

Flax Command Line Help:

Usage: runflax <source file> <optional switch>

Switches:
      -A, -About
      -E, -Encrypt
      -I, -Interactive
      -V, -Version
      -L, -License
      -H, -Help
      -C, -Compile

Enter 'runflax' <switch> for further information on that switch.
```

COMPILING

Assuming that you've set up Flax in its recommended default configuration, then compiling your code is a simple matter of entering the '**runflax**' command in the terminal, followed by the name of the file to be compiled, and finally the '**-c**' switch to indicate that you only wish to compile your code to check for errors.

Command Line Switches

For test compiling your code only the '**-c**' switch is required.

```
runflax helloworld.flax -c
```

Inline Switches

Within your code, however, there are a number of compiler switches you can add to provide additional detail or to control the output of the compile process. None of these inline switches are mandatory and can be added or removed at will.

Inline switch	Description
@compileSnapShot	This switch is used to tell the compiler to display a minimal snapshot of your program's compiled data. The minimal report only includes the '**variable symbol table**', '**program counter**' and '**irq**' addresses. The '**irq**' address will be ignored if no IRQ is configured. This is a compile-only switch and is ignored when you run the program.
@compileSnapShotFull	This switch is used to tell the compiler to display a complete snapshot of your program's compiled data, including code, variables, constants, and all data stores. This is a compile-only switch and is ignored when you run the program.
@runtimeSnapShot	This switch is used to tell the runtime to provide a minimal runtime snapshot after execution. This snapshot allows viewing of the final state of the '**registers**' and '**program counter**' position.
@runtimeSnapShotFull	This switch is used to tell the runtime to provide a detailed runtime snapshot after execution. This snapshot allows viewing of the final state of '**variables**', '**registers**' and the contents of both the '**general stack**' and '**execution stack**' as well as all other data stores.
@suppressWarnings	As the name suggests, this switch forces the compiler to hide any warnings in its output. Warnings are generally not considered serious enough to stop execution of your program, though in best practice they should still be addressed. **This is a compile-only switch and is ignored when you run the program.**

ENCRYPTING YOUR FLAX CODE.

Flax has the ability to encrypt your flax program if you wish to distribute it without others being able to copy your code. To do so, you just need to use the '**-e**' switch, which will encrypt your file then create a new file with the '**.eflax**' extension.

```
runflax helloworld.flax -e
```

There is **no** way to decrypt the file, so make sure you keep a copy of your original Flax code should you wish to make further changes and improvements.

Before encryption, you should also make sure that your code has no errors, as the encryption process does not compile and check your code as part of the process.

RUNNING YOUR PROGRAM

Running your Flax program is as simple as compiling. If you run your program with just the '**runflax**' command followed by your filename, your code will be automatically compiled and run within the terminal in normal terminal blocking mode where a '**return**' is required after data entry. If your program doesn't require user input and is just outputting the result of calculations, then this is a perfectly valid option. If, however, your program needs to respond in real time to keypress and other events, then running your program in interactive mode with the '**-i**' switch, discussed below, is the better option.

```
runflax helloworld.flax
```

Command Line Switches

The only command-line switch available when running your program is the '**-i**' switch, used to place the terminal in interactive mode so that it can respond to keyboard and event input without the normal terminal blocking behaviour.

```
runflax helloworld.flax –i
```

Programs run in interactive mode perform considerably better than when the terminal is run normally. If peak performance is your requirement, then this is the better option.

The '**sys**' instruction, discussed later in this manual, is used to adjust runtime performance as well as enabling or disabling additional features. The '**sys**' instruction also differs from compiler in-line switches in that it can be repeatedly called within your running application to provide intimate control over the runtime experience.

Interactive Mode

As mentioned above, interactive mode is when the terminal is placed in non-blocking mode so that it can respond to events in real time without the normal terminal blocking behaviour. The **'-i'** switch is required when running your program to enter this mode.

When your program exits, the terminal will be returned to its normal state.

Shell Scripts

The '**runflax**' command is in fact a shell script wrapper for the actual Flax executable. It's possible to run your programs by just entering '**flax**' and the filename, though running Flax in this manner causes the '**-i**' switch to be ignored.

It is the '**runflax**' script that intercepts the '**-i**' switch and places the terminal in non-blocking mode. It is therefore recommended that '**runflax**' always be used as the normal method of execution.

EDITING CODE

Flax files are purely text-based, so any program capable of editing and saving text files would be a useable choice. Even '**TextEdit**' on macOS would be suitable, though a more advanced editor with line numbers and the option to highlight keywords and integrate scripts, which allow you to run or compile your code directly, would be a preferable option.

EDITOR CHOICE

In-house, we recommend '**BBEdit**' from '**Bare Bones Software**'. It is a macOS-only application and provides a huge range of customisable options. It is lightweight, fast, and also provides full support for automation, integration scripts, and code-less language modules, allowing it to be configured as a bespoke development environment.

Other popular editors such as '**VSCode**' or '**Sublime Text**' would also be suitable.

If you wanted to stay completely in the terminal though, '**THE**' or '**The Hessling Editor**', available through '**Home-brew**' or directly from 'https://hessling-editor.sourceforge.net/' is an excellent choice.

INTEGRATION SCRIPTS

On release, we'll be providing integrations and automation scripts for BBEdit. In time, we hope that other users will provide scripts and language modules which can be added to our GitHub repository.

LANGUAGE MODULES

At their simplest, language modules provide highlighting of reserved words and special pre-defined constants. We'll be including a code-less language module for Flax, usable in BBEdit and other editors which support the technology.

In time, as with integration scripts, we hope that other users might provide their own.

STYLE GUIDE

The nice thing about assembly is there is very little in the way of formatting requirements. There are no indenting rules or start and end blocks, with the exception of terminating a subroutine with an '**rts**' instruction.

Normal practice is to always have comments, variables, and labels left-justified and operational instructions indented with a single tab. Even when you have nested loops, unless you feel it necessary for readability, nested loops are not indented any differently.

There is nothing stopping you, however, from laying out the code however you wish. The only strictly enforced rule is that only a single instruction per line is allowed, and the compiler will complain if you try.

```
;Example FileIO File Copy Program.
_start:
      lda #inFile
      in fileIO, openFile

      lda #outFile
      out fileIO, createFile

fileCopyLoop:
      lda inFile
      jl endCopy ;Checking for EOF -1
      sta outFile
      jp fileCopyLoop

endCopy:

      lda #inFile
      in fileIO, closeFile

      lda #outFile
      out fileIO, closeFile

      brk

file inFile, "files/test.txt"
file outFile, "files/out.txt"
```

Suggested Formatting.

The above code is a good example of well-formatted code. All the labels are left-justified, the instructions indented, and the variables defined at the end of the program.

If the program also included subroutines, these would normally be placed after the main loop and after a '**brk**' instruction to prevent program counter overrun, which is where the program counter overruns a subroutine without the subroutine being called.

YOUR FIRST PROGRAM

It is something of a tradition when learning a new programming language that the first program you write is a 'Hello World' application, or in Flax's case a 'Hello, Flax World!' program.

In this section, we provide three methods of achieving this result as well as a novel method of printing the string backward. The first example uses a more traditional assembly approach using a pointer as an offset to read each byte in the variable's memory block. The second approach uses a loop after reading the length of the message variable, and the third an even simpler method by simply passing the contents of the variable's memory block to '**stdout**'.

So the next step is to open up your favourite editor and get typing. *(Please don't copy and paste! That would be cheating.)*

HELLO FLAX WORLD!

The first version is a traditional pointer-based version where we use the value stored in the '**b**' register to iterate through the individual characters of the string until we encounter a null character or a '**0',** at which point we use the '**jz**' *(Jump on Zero)* instruction to jump to the end label.

Quoted strings in Flax are automatically terminated with a null character.

```
;Hello, Flax World! - Pointer Example
_start:
      lda 0
      sta b

mainLoop:
      lda myMsg, b
      jz end
      out
      lda b
      inc
      sta b
      jp mainLoop

end:
      brk

db myMsg, "Hello, Flax World!\n"
```

Traditional Assembly Pointer Based Solution.

The second version uses a loop after we first get the length of the variable with the '**len**' instruction. '**stc**' stores this value in the '**c**' register, which is used as the loop counter. As loops in Flax always count down from the value given to zero, we need to invert the value of '**c**' by subtracting it from the total length. If we didn't, the message would print in reverse order, so if that were your desire, you could remove any reference to '**b**', the '**sub**' instruction, and simply use the value of '**c**' as the pointer. In this case though, and for ease of use, we've stored the length in the '**b**' register so that we don't have to continually use the '**len**' instruction to get the length.

With this version, we also don't require an 'end:' label, though we should still add a '**brk**' instruction as it's good practice.

```
;Hello, Flax World! - Loop Example
_start:
      len myMsg
      sta c
      sta b

mainLoop:
      lda b
      sub c
      lda myMsg, a
      out
      lp mainLoop

      brk

db myMsg, "Hello, Flax World!\n"
```

Simpler Loop Version, using the '**len**' instruction.

```
;Hello, Flax World! - Reverse Loop Example
_start:
      len myMsg
      sta c

mainLoop:
      lda myMsg, c
      out
      lp mainLoop

      brk

db myMsg, "Hello, Flax World!\n"
```

Loop Version, printing in reverse.

The last example is the simplest of them all. It doesn't require any of the complexity of the others, as we simply pass the entire string buffer to '**stdout',** the standard output stream.

```
;Hello Flax World! - Using STDOUT
_start:
      out msg
      brk

db msg, "Hello, Flax World!\n"
```

Simplest version using '**stdout**'.

REFERENCE

With its limited instruction set, Flax may at first appear simple in nature, but as with most programming languages, there is an infinite variety in end-user program design and the instructions used. This reference section seeks to detail Flax's usage and potential pitfalls.

CASE MATTERS

When editing code for Flax, regardless of your choice of editor, case matters. Everything in the Flax language is case-sensitive, including the naming of your variables, labels, and even such insignificant things like the inline compiler switches.

The ***one and only*** exception to this rule is the optional command-line switch you might place after the '**runflax**' executable and your source code filename.

CODE PROFILING

Code profiling is the process of examining your code given a set of rules which could detect issues not found with normal syntactical checks or instruction pattern matching.

Flax's compiler makes use of all three approaches and with its profiler further attempts to identify issues which could result in performance, stability or even single-out items that don't conform to best practice.

It would be exceedingly difficult to explain the exact process and rules the code profiler uses, but it's comforting to know it's there as a crucial part of the compile process.

Although there are a number of compiler switches which refine its process none has the ability to disable any of the three core methods we employ during compilation.

COMMENTS

Flax has support for both full-line comments and post-instruction, or inline, comments placed on the same line as the instruction. There is '**no**' support for larger block-style comments within an opening and closing tag.

Regardless of the style of comment used, all are prefixed with the '**;**' character, which is somewhat of a tradition in assembly language.

```
;This is a line style comment
_start:
```

Line Comment.

```
_start:
      lda 10 ;This is an inline comment
```

Inline Comment.

COMPILER FLAGS AND SWITCHES

The table below lists all the compiler flags and switches, their usage, and function. Command-line switches are those which are placed on the command line with the '**runflax**' command, filename, and optional switch. Inline switches and flags are those that are placed directly within the program's code to further refine the compiler configuration.

Switch	Type	Description
-c	Command Line	Restricts Flax to only compile the code and display any errors or warnings. The '**-c**' switch overrides all other command-line switches.
-e	Command Line	Encrypts the passed flax file, saving it with the '.eflax' extension. There is no way to decrypt the file, so make sure you keep a copy of your original flax source if you want to make later changes.
@compileSnapShot	inline	This switch is used to tell the compiler to display a minimal snapshot of your program's compiled data. The minimal report only includes the '**variable symbol table**' and '**system label**' positions. This is a compile-only switch and is ignored when you run the program.
@compileSnapShotFull	inline	This switch is used to tell the compiler to display a complete snapshot of your program's compiled data, including code, variables, constants, and all data stores. This is a compile-only switch and is ignored when you run the program.
@runtimeSnapShot	inline	This switch is used to tell the runtime to provide a minimal runtime snapshot after execution. This snapshot allows viewing of the final state of the '**registers**' and '**program counter**' position.
@runtimeSnapShotFull	inline	This switch is used to tell the runtime to provide a detailed runtime snapshot after execution. This snapshot allows viewing of the final state of variables, registers and the contents of both the general stack and execution stack as well as all other data stores.
@suppressWarnings	inline	As the name suggests, this switch forces the compiler to hide any warnings in its output. Warnings are generally not considered serious enough to stop execution of your program, though in best practice they should still be addressed. This is a compile-only switch and is ignored when you run the program.

CONSTANTS

There are three styles of constants within the Flax language: numeric constants or numbers, either floating-point or integers; generic constants such as true or false; and worded constants, which are constrained to be used with a specific instruction and cannot be used elsewhere. Regardless of the style of constant, all will be replaced with their numeric equivalent during compilation.

Generic Constants

Constant	Value	Description
false	0	Provides a readable replacement for the numeric representation of false, providing increased clarity to your code.
pi	3.1415926535	Provides a readable replacement for the numeric value of 'PI'.
true	1	Provides a readable replacement for the numeric representation of true, providing increased clarity to your code.

All constants in this table can be used with any instruction which accepts a normal constant.

Instruction Specific Constants (words)

Constants listed in this section are what are referred to in the Flax language and compiler as worded constants or words, which can only be used with a specific instruction. Words are one of the things which make Flax's compiler so capable. Rules within the compiler strictly enforce the usage of these words to instruction as well as parameter position. Even if a numeric constant of the same value was used as a replacement, the compiler would still flag it as an error.

Using the constant name rather than the value, or what is referred to as a magic number, makes your code more readable and quickly understandable should you come back to it later.

Constant	Value	Type	Used By	Operand	Usage
autoLineFeedEnabled	4	flag	sys	2	Controls the enabled state of line feeds after numeric print operations. Overrides the '**autoSpaceEnabled**' flag.
autoPauseEnabled	3	flag	sys	2	Controls the enabled state of the auto pause system during yield phases.
autoPauseTime	2	value	sys	2	Controls the setting of the auto pause time key value pair.
autoSpaceEnabled	5	flag	sys	2	Controls the enabled state of space separation during numeric print operations. This is overridden by the '**autoLineFeedEnabled**' flag.
debugEnabled	1	flag	sys	2	Controls the enabled state of displaying debug information after program execution.

Constant	Value	Type	Used By	Operand	Usage
divideByZeroErrorEnabled	8	flag	sys	2	Controls the enabled state of detecting and displaying an error on divide or modulus divide by zero.
flag	1	switch	sys	1	Places the '**sys**' instruction in '**flag**' setting mode.
floatingPointPrecision	3	value	sys	2	Controls the setting of the floating-point precision key value pair. This setting is only used when printing floating-point values to the console. Internal operations utilise, however, may points of precision are required.
hltPollingTime	4	value	sys	2	Controls the setting of the '**hlt**' instruction keyboard / sleep polling frequency.
irqEnabled	2	flag	sys	2	Controls the enabled state of the IRQ system.
irqPerformance	1	value	sys	2	Controls the setting of the irq performance key value pair.
number	0	switch	out	1	Forces the '**out**' instruction to always output the value within the '**A**' register as a number regardless of value.
sysPerformance	0	value	sys	2	Controls the setting of the system performance key value pair.
timerTimeNow	0	switch	gt	1	Instructs the '**gt**' instruction to get the current Unix time in milliseconds.
timerTimeSince	1	switch	gt	1	Instructs the '**gt**' instruction to calculate the difference between the now and since values.
value	0	switch	sys	1	Places the '**sys**' instruction in '**value**' setting mode.

Constants in this table can only be used with the instruction listed.

Table Index:

- '***Switch***' *is a value which controls the internal operation of an instruction.*
- '***Flag***' *denotes that the constant is used for setting a true/false condition.*
- '***Value***' *indicates the constant represents a key value pair.*
- '***Operand***' *denotes the parameter position on the line of code after the instruction.*

ERROR MESSAGES AND WARNINGS

Flax's compiler is extremely thorough in catching most errors during compile, chiefly thanks to its custom-designed pattern matching system, strict instruction-to-constant rules, and code profiling. This in-depth approach results in an extensive list of error messages and warnings, which after a little practice, you'll learn to catch yourself and easily avoid.

Compile Errors

- **_start: label not found.**
  Every Flax program requires a '**_start:**' label to indicate when in the code the program should start executing and the program counter set accordingly. In most cases, the start label is normally placed at the start of the program, though this is not necessary. The '**_start:**' label can be placed anywhere in your code, and the program counter will be set accordingly.

- **Not a printable string variable type..**
  Indicates that the passed variable is not a directly printable type.

- **Invalid Pattern.**
  This is the normal output by the compiler when the pattern constructed for the line of code being compiled has not been found in the allowed patterns library.

- **Jumping to the _irq: label is disallowed.**
  This error might seem odd at first, to have a label in your code which you can't navigate to. The '**_irq:**' label though is unique. You certainly define it and place whatever code you wish within the IRQ block, but after that, the IRQ code is handled entirely by the system. During a yield phase and within the appropriate IRQ frequency settings, the system pauses execution of your main code to execute what is contained in the IRQ block. When finished and terminated with an '**rti**' instruction, your primary code will continue from where it was previously. It's easier to think of the IRQ like a background thread where you occasionally run specific code not easily handled elsewhere in your program.

- **Invalid or duplicate label name.**
  Not much to say here. The specified label does not meet label standards or a label of the same name already exists. A normal label must be a single word suffixed with a colon and cannot start with a number, underscore, or special character.

- **Only the '_start:' and '_irq:' system labels may begin with an underscore.**
  The error is self-explanatory; only the two system labels '**_start:**' and '**_irq:**' are allowed to begin with an underscore. This restriction has been placed on non-system labels so that system labels are more easily identified.

- **Invalid compiler switch.**
  As the error suggests, the '**@**' prefixed switch name does not match a known switch. Everything in Flax coding is case-sensitive, so if it is a valid switch, it may be spelt or cased incorrectly. You could easily tell though; if you're using an editor with Flax highlighting capabilities, then the spelling or case issue would be easily identifiable.

- **Invalid variable definition.**
  The variable definition does match the correct variable definition syntax. This will usually occur if you've neglected to give the variable a default value.

- **Invalid or duplicate variable name.**
  Just like label names, variables have the same naming standards. Additionally, a label or variable name cannot match a reserved word, constant, register name, contain invalid characters, or match an existing variable name.

- **Invalid byte data.**
  The byte variable type can only contain unsigned byte values between 0 and 255 (inclusive). A single byte value, a list of numbers, and even a quoted string must meet this requirement. If you need signed values which allow for negative numbers, then you should use the '**dw**' declaration for a signed integer type.

- **Invalid integer data.**
  The same as above. The integer data provided is not valid. This usually occurs when accidentally using '**dw**' to define a float value rather than '**dd**' to define a double, or attempting to assign a quoted string to the integer type.

- **Invalid float data.**
  Exactly the same as the above except for an float/double type variable '**dd**'.

- **Memory block size must be a positive integer > 0.**
  When defining a memory block of any size, the value specified must be an integer greater than zero. It's pretty hard to create a memory block of a negative size.

- **Invalid Quoted String.**
  This error usually occurs when the string value is not properly quoted. It must start and end with a quote character and when evaluated must contain exactly two quotes. If you want to have quotes in the string when printed to the screen, you need to escape any quotes within the string with a backslash character, e.g.: \" You can also do the same for tabs and line feeds: \t or \n.

Compile Profiling Errors

- **No 'rti' instruction found with '_irq:' label set.**
  Unlike a normal subroutine, the '**_irq**' subroutine must be terminated with an '**rti**' *(return from interrupt)* instruction. There is a significant difference in the clean-up process when the interrupt code terminates, hence the requirement for an instruction other than '**rts**'.

- **'rti' instruction found without '_irq:' label set.**
  During compilation, no '**rti**' instruction was found with the '**_irq:**' label set. This could indicate that it's been forgotten or an '**rts**' instruction has been used instead.

- **'rti' instruction found before '_irq:' label.**
  Indicates that the '**rti**' instruction was found earlier in the code than the '**_irq:**' label, so the IRQ subroutine has not been properly terminated.

- **Multiple 'rti' instructions found. Only one allowed when '_irq:' label is set.**
  As there is only one IRQ subroutine allowed, it's reasonable to surmise that only one '**rti**' instruction is allowed as well. If multiple '**rti**' instructions have been found, then clearly there are some logic or syntax issues present.

- **No 'rts' instructions found though subroutine jumps detected.**
  This is the inverse of the error below. If there have been subroutine jumps detected, then there should be corresponding '**rts**' instructions.

- **'rts' instruction found with no subroutine jumps detected.**
  As with the '**rti**' instruction, if there have been no subroutine jump instructions found, then there should be no '**rts**' instructions either.

- **Label name specified on 'lp' instruction not found before loop.**
  The label name specified on the '**lp**' instruction has not been found before the loop statement. Flax only supports back facing loops so the loop label must be declared before the '**lp**' instruction.

- **'rts' Instruction found within the '_irq:' subroutine.**
  An '**rts**' instruction has been found within the IRQ subroutine, as Flax does not supported nested subroutines this logic is not allowed.

Compile Warnings

There are a few warnings generated during compilation. Assembly is hard, and any mistakes found in code are usually considered serious, so warnings are rare. Most warnings are based on code profiling to identify issues that can't be detected with pattern matching or syntactical checks.

- **Difference in subroutine calls to 'rts' instructions. Suggests unbalanced calls to 'rts' usage.**
  This warning is pretty self-explanatory. During code profiling, a significant difference has been found in the number of jump instructions to corresponding return statements. This may be perfectly valid within the scope of your program. The compiler is not capable of reading your mind. It's just a warning that a difference has been found. The default variance allowed before this warning is displayed is five.

- **Multiple 'rts' instructions found without intervening labels..**
  This warning indicates that multiple '**rts**' instructions have been found without intervening labels, which is not considered good practise. If the program counter was to overrun the first then execute another a possible execution stack underflow could occur resulting in a system error.

Runtime Errors

Any errors generated while running will commonly result in the program being terminated and with the graceful exit flag set to false. To help with such occurrences, Flax's compiler builds a symbols table that allows even the compiled runtime code to indicate the line number in the source which resulted in the error. This makes tracking down bugs in your code simpler and with fewer headaches.

- **Invalid program counter address.**
  Indicates that the program counter was not set correctly during compile. This error will probably never appear as the compiler checks for a valid start label.

- **Invalid IRQ address.**
  Similar to the previous, it indicates that the IRQ address has not been set correctly. Again, it is unlikely to appear due to compiler checks and profiling.

- **Divide by zero error.**
  Indicates that a divide by zero condition has been detected during a divide operation. This error can be disabled by setting the flag '**divideByZeroErrorEnabled**' to '**false**' with the '**sys**' instruction. See the '**Key Value Pairs and Flags**' section of this manual.

- **Modulus divide-by-zero error.**
  Indicates that a divide by zero condition has been detected during a modulus operation. This error can be disabled by setting the flag '**divideByZeroErrorEnabled**' to '**false**' with the '**sys**' instruction. See the '**Key Value Pairs and Flags**' section of this manual.

- **Execution stack overflow.**
  Indicates that the execution stack is full. This stack holds return addresses for when returning from subroutines. If this error appears, it is likely you have circular jump logic in your code.

- **Execution stack empty.**
  This error will appear if the execution stack is empty when either of the two above instructions is executed. Possible causes are unstructured jumps into a subroutine or accidental program counter overruns by not placing an exit catch before all the subroutines, or a called subroutine is missing a return statement.

- **Stack overflow.**
  The opposite of the above. This error occurs when the stack is full and is most frequently caused by neglecting to remove values after pushing them onto the stack.

- **Stack empty.**
  Indicates there were no values on the stack. Most often caused by neglecting to back up and restore modified registers in a subroutine or simply forgetting to '**push**' a value onto the stack expected in a later call to a subroutine.

- **Invalid value for key-value pair.**
  Occurs when attempting to set a key-value pair with the '**sys**' instruction, and the value is not within the allowed range.

- **Invalid 'flag' setting. IRQ state cannot be altered when no '_irq:' subroutine exists.**
  This is displayed when trying to adjust the IRQ enabled state when no IRQ subroutine has been defined.

- **Invalid 'flag' setting for true/false condition.**
  The value given does not match the numeric equivalents for true or false: '**0**' or '**1**'.

- **Invalid 'value' setting. IRQ state cannot be altered when no '_irq:' subroutine exists.**
  This is displayed when trying to adjust the IRQ performance when no IRQ subroutine has been defined.

- **Offset outside bounds of variable.**
  Occurs when using '**lda**' or '**sta**' to read or write to a variable and the given offset is outside the bounds of the variable's memory block.

- **Insufficient Stack Space.**
  Advises that there is insufficient stack space available for the current operation.

- **Stack size mismatch.**
  This indicates that there is insufficient entries on the stack for the current operation.

- **Not a valid terminal control sequence ID.**
  Indicates that the value loaded into the '**A**' register is not a valid terminal control escape sequence ID.

- **Not a valid variable address.**
  Indicates that the address loaded into the '**A**' register is not a valid variable address.

- **Not a valid variable type and or size.**
  Indicates the the variable address loaded into the '**A**' register is not of the required type or size for the current operation.

- **Not a valid integer value.**
  Indicates that value loaded into the '**A**' register is not a valid integer (whole number) required by the current operation.

- **File already open.**
  Advises that the file has already been opened by another process within your code.

- **File not found.**
  Advises that the specified filename has not been found in local directory or subdirectory.

- **File has not been opened for read.**
  Occurs when you attempt to print the contents of a file to the screen but the file has not been opened in read mode.

- **File is empty and cannot be opened for read.**
  The file on the disk contains no information so a memory buffer for the file could not be created.

- **File has not been opened for write.**
  Indicates that the file you are attempting to write to has not been opened in write mode.

- **Error reading from file.**
  Displayed when there is a system error reading from the specified file. In such cases you should check you have the necessary permissions to read from that location.

- **Error creating file.**
  Occurs if there was an error creating a file. File system permissions is the normal reason for this error to occur.

- **Error appending to file.**
  The same as the above error. Check that you have permissions to the area you are attempting to write to.

General System Errors:

This section lists any general system messages which relate to the entire system and are not limited to just the compiler or runtime.

- **Source file not found.**
  Reasonably obvious that the given Flax file has not been found.

- **Source file is empty or failed loading.**
  Along with the general system message for the same. The Flax file is either empty and contains no code, or there was an error loading the file, in which case you should check that you have the required permissions to the file.

- **File appears encrypted but has the '.flax' extension.**
  On examination, the source code does not contain any of the search phrases used to determine encryption. If your file is empty or you haven't coded anything yet this might occur. It is important that you do not encrypt your file until complete and you intend to distribute it.

- **File is not encrypted but has the '.eflax' extension.**
  The '**.eflax**' source file provided contains unencrypted elements yet has the '**.eflax**' extension. Please rename it to '**.flax**' to correct this. If this was done in an attempt to decrypt the code, it is not possible.

- **Invalid file extension for this operation.**
  Indicates that you are trying to encrypt a file which doesn't have the correct extension. Only '**.flax**' files are allowed during this process.

- **File appears to be encrypted already.**
  On examination of the file it appears to already be encrypted. Encrypting it again would cause irreparable damage.

- **Error: Invalid key value pair.**
  Indicates that values within the values library do not fall within the specification of min/max integer values.

- **Library file empty or failed loading.**
  Loading the specified library file returned no information, which may indicate the file was empty or failed to load.

- **Library file not found.**
  The specified library file was not found in the libraries directory.

- **Incompatible file type.**
  Indicates that the file passed to '**runflax**' or the '**flax**' executable does not have a '**.flax**' or '**.eflax**' extension and cannot be safely verified as a Flax program.

FILE IO

Flax has inbuilt file input and output capabilities, though to adhere to file system security rules, file access is limited to the current directory or a subdirectory of where your flax files are stored.

To read from or write to a file, you first need to create a '**file**' type variable which includes a quoted string with the filename. The file does not need to exist for the variable to be created, as you may be wishing to create and write to a new file at runtime.

```
file variableName, "test.txt"
```

Creating a file variable.

When opening a file for reading, however, the existence of the file will be checked at runtime and an error displayed if not found.

```
lda #variableName
in fileIO, openFile
```

Opening a file for reading.

```
lda #variableName
out fileIO, appendFile
```

Opening a file for appending.

When opening a file for reading you use the '**in**' instruction, for output you must use the '**out**' instruction. You cannot have the file opened for read or write at the same time.

Constant	Value	Type	Purpose
fileIO	3	port	This is the identifier of the '**fileIO**' port within the virtual port system used for all '**fileIO**' operations.
openFile	0	switch	Used when reading from a file to open the file and read its contents into the internal memory buffer. **The file must exist on open.**
closeFile	1	switch	Used when reading from or writing to a file. It closes the file and clears the memory buffer. When closing a new file or one opened for append, the contents of the memory buffer are written to the file.
appendFile	2	switch	Opens a file in append mode. When closed, the memory buffer will be appended to the existing file.
createFile	3	switch	Opens a file in create or overwrite mode. When closed, the memory buffer will be written to the new file or overwrite the old one.
purgeFile	4	switch	Discards any changes made to the memory buffer and closes the file. No change is made to the file on disk.

Constant	Value	Type	Purpose
resetPointer	5	switch	Resets the file read or write pointer to '**0**'. When reading from a file, this simply allows the memory buffer to be read again. When writing, however, if the pointer is moved to an earlier position, then all values after this point will be lost. If moved forward, then the intervening space will be filled with null '**0**' characters.

All operators require the file variable's memory address to be loaded in the '**A**' register with the **#variableName** syntax.

Once a file is opened, all file read and write operations are performed with the '**lda**' and '**sta**' instructions exactly like any other variable type. The only difference is that if an '**offset**' is provided, the file pointer will be moved to that point with the same considerations listed for '**resetPointer**'.

```
;Reading a file without an offset
      lda variableName

;Reading a file with and offset
      lda variableName, 10
```

Reading from a file.

```
;Writing to a file without an offset
      sta variableName

;Writing to a file with an offset
      sta variableName, 10
```

Writing to a file.

When reading from a file, a '**-1**' will be returned when you have reached the end of the file buffer. At this point, you can either close the file or reset the file pointer with '**resetPointer**' or using '**lda**' with an offset of '**0**'. Doing so will allow the file buffer to be traversed again or as many times as you like.

A point to note is that no change is made to the file on disk until you close the file within your Flax code. If at any point you decide you don't want to write the changes, you could use '**out fileIO, purgeFile**' to discard any changes.

FREQUENTLY ASKED QUESTIONS

Below are answers to frequently asked question regarding Flax, its language and syntax, licensing, editor configuration and other areas of interest.

What does the name Flax mean?

Flax is not an acronym. It's just a name we chose which sounded good but at the same time honours the flax plant, which in prior times was used for making cloth and sails for sailing ships. The flax plant has so many beneficial uses that if used to its full potential could be advantageous to all. If you require an acronym though, '***F***unctional ***L***earning ***A***ssembly E***x***ecution' would fit.

What operating systems does Flax support?

Flax is only supported on macOS, with Apple Silicon or x86 processors.

Could Flax be used to write a virus?

No! Flax has been designed to adhere to all operating system security requirements. It will never try to access any area outside which you've given it access to, and that list should only include the folder you've chosen to store your Flax files within. Flax does not and never will have network or internet access capabilities. Even its FileIO is limited to the directory in which you've stored your Flax code.

What can Flax be used for?

Anything you wish. From cluster computing tasks, general programming, personal or structured education. This list is endless, really. You could even just have a play and try something different. Flax was designed to promote inventive thinking and creative solutions.

How much does Flax cost?

Flax has a non-commercial use license, so for personal, non-business use, it's free. Business use will be assessed on a case-by-case basis, but at most a small yearly per-seat license fee may be required. You're welcome to email us for further information.

FLAGS AND KEY VALUES PAIRS

Flags and Key Value Pairs are values or settings within the Flax runtime which can be changed at any time within your active program to give far greater control over active features and system performance.

Regardless of which, flags or keys, are all modified with the '**sys**' instruction with the appropriate worded constant.

A point to note: attempting to set either a Flag or Key Value Pair to an incorrect value will result in a system error. We could just assume the default instead, but hiding mistakes does not promote learning.

Flags

Flags are settings which are controlled with a simple true/false condition. Auto pausing or enabling or disabling the IRQ system are two such examples.

Constant	Default	Purpose
autoLineFeedEnabled	false	Controls the enabled state of line feeds after numeric print operations.
autoPauseEnabled	false	Controls the enabled state of the auto pause system during yield phases.
autoSpaceEnabled	true	Controls the enabled state of space separation during numeric print operations.
debugEnabled	false	Controls the enabled state of displaying debug information after program execution.
divideByZeroErrorEnabled	true	Controls the enabled state of detecting and displaying an error on a divide or modulus divide by zero.
irqEnabled	false	Controls the enabled state of the IRQ system.

To modify any of the setting in this table you first need to load true or false in the '**A**' register

```
lda true
sys flag, debugEnabled
```

Setting a runtime flag.

Key Value Pairs

Key Value Pairs are settings which require a value within a specific range and, as such, provide granular control over settings, including system performance or floating-point precision. All have a minimum and maximum setting as well as a recommended default which falls within the allowable range.

Constant	Minimum	Maximum	Default	Usage
autoPauseTime	5	500	20	Controls the time the runtime thread sleeps during a yield phase if the flag 'autoPauseEnabled' is set to true.
floatingPointPrecision	1	10	2	Controls the number of decimal places that are printed with a floating-point number when output to the console.
hltPollingTime	10	100	20	Controls the time the '**hlt**' instruction sleeps the main runtime thread before checking the keyboard for a keypress to resume program execution.
irqPerformance	20	100	20	Control the number of yield phases which have to occur before the IRQ system will trigger. This setting is also reliant on the IRQ system having been properly configured.
sysPerformance	5	50000	1000	Controls the number of assembly instructions executed before a yield phase occurs. Higher numbers offer greater performance, though at the cost of higher CPU loads.

To modify any of the setting in this table you first need to load the new value into in the '**A**' register

```
lda 40
sys value, autoPauseTime
```

Setting a Key Value Pair.

INSTRUCTIONS

Instruction Set

Flax has been designed with a RISC-based architecture in mind. *(Reduced Instruction Set Computer.)* Its instruction set includes only forty-four parent instructions *(listed below)*, which, with variations of parameters, equates to a total of seventy-eight functional opcodes or patterns. *(Their purpose, parameters, and syntax are detailed later in this manual.)*

```
abs, add, brk, cmp, csp, dec, div, gt, hlt, idiv, in, inc, jg, jgz, jl, jlz, jnz,
jp, jsr, jsrnz, jsrz, jz, lda, len, lp, mod, mul, neg, nop, out, pop, popa, push,
pusha, rnd, rti, rts, sqrt, sta, stb, stc, sub, sys, xch
```

Flax Instruction Set

Such a reduced instruction set may at first appear limiting, but one of the goals of Flax is to teach functional programming concepts, which often require inventive solutions. A reduced instruction set is not a limitation, though a lack of imagination and critical thinking is.

This list of instructions may appear short, but it was chosen with care. There is nothing that can't be done with creative use and careful consideration. RISC processors, by design, often require more instructions to be run to complete a similar task than their bigger brother, the CISC processor *(Complex Instruction Set Computer)*, but generally, they perform quicker.

Each instruction, or pattern to clarify further, is a completely separate opcode on the CPU, which is just a fancy way of saying what part of the chip gets turned on when a particular instruction is run.

Each of them does one thing and one thing only, and they do that one thing extremely fast.

Parameter Types

Instructions within Flax are often capable of accepting a number of different parameter types. Detailed below is the list of parameter types, and in the next section, a detailed reference of their allowed usage within Flax's instruction set.

Parameter Type	Description
address	The address of a variable loaded in the '**A**' register using the '**#variableName**' syntax.
constant	A positive or negative numeric value, either a byte, integer, floating point, or a generic constant such as '**true**', '**false**', or '**pi**'.
word	A worded constant defined in the constants library, which will be replaced with its numeric equivalent during compilation. Words are normally constrained to only function with specific instructions.
label	A label name without the ':' suffix defined within the code and used by structured or unstructured jump instructions.
register	One of the named system or general-purpose registers.
variable	A named variable of any type defined in code using the correct variable declaration syntax.

Syntax, Patterns and Variations

This reference section documents all instructions and their valid patterns, usable within a Flax program.

ABS	**Absolute**
Description:	Replaces the value held in the '**A**' register with its absolute value regardless of sign.

Allowed Patterns / usage:		
abs	Replaces the value held in the '**A**' register with its absolute value regardless of sign.	Bitcode: **72**

ADD	**Add**
Description:	Used to add the passed value to the value held in the '**A**' register.

Allowed Patterns / usage:		
add constant	Adds the value of the passed constant to the value held in the '**A**' register.	Bitcode: **51**
add register	Adds the value held in the passed register to the value held in the '**A**' register.	Bitcode: **52**
add variable	Adds the first value in a variable's memory block to the value held in the '**A**' register. If you need to perform this action with a value held in a later offset of the variable memory block, you'd first need to load that value with the '**lda variable, offset**' instruction and store that value elsewhere before proceeding.	Bitcode: **53**

BRK	**Break**
Description:	Terminates the program by setting the '**runable**' flag to false.

Allowed Patterns / usage:		
brk	Terminates and exits the program by setting the internal register '**runable**' to false	Bitcode: **1**

CSP	**Clear Stack Pointer**
Description:	Clears the stack pointer, effectively erasing all values held on the stack.

Allowed Patterns / usage:		
csp	Clears the stack pointer, effectively erasing all values held on the stack.	Bitcode: **68**

Items of Note:	The '**csp**' instruction is an extremely '**DANGEROUS**' instruction. By using it, you are effectively erasing all values currently held on the stack. This can be problematic and could result in stack underflow conditions should there be a later instruction expecting a value to be present. With good program design, there should be no reason this instruction would ever be required.

CMP	**Compare**
Description:	Used to compare the number being passed to the value held in the '**A**' register.

Allowed Patterns / usage:		
cmp constant	Compares the passed constant with the value in the '**A**' register.	Bitcode: **21**
cmp register	Compares the value held in the passed register with the value in the '**A**' register.	Bitcode: **22**
cmp variable	Compares the first element of a variable's memory block with the value held in the '**A**' register. If you need to perform this action with a value held in a later offset of the variable memory block, you'd first need to load that value with the '**lda variable, offset**' instruction and store that value elsewhere before proceeding.	Bitcode: **23**

Items of Note:	A comparison works by subtracting the value in the '**A**' register from the value being passed. This would result in the difference being loaded back into the '**A**' register, and the original value being lost. You need to back up the value in the '**A**' register first if you need to perform repeat comparisons, such as checking key codes. A variable or using the '**R0**' or '**R1**' registers is an acceptable solution. If the resulting value in the '**A**' register is '**0**', it indicates that both values were the same, equal. The '**jz**' or '**jnz**' instructions could be considered as '**Jump when Equal**' or '**Jump when not Equal**'.

DEC	**Decrement**
Description:	Decrements the value in the '**A**' register by one.

Allowed Patterns / usage:		
dec	Decrements the value in the '**A**' register by one.	Bitcode: **15**

DIV	Divide
Description:	Used to divide the value held in the '**A**' register with the passed value.

Allowed Patterns / usage:		
div constant	Divides the value held in the 'A' register with the value of the passed constant.	Bitcode: **60**
div register	Divides the value held in the '**A**' register with the value held in the passed register.	Bitcode: **61**
div variable	Divides the value held in the '**A**' register with the first value held in the passed variable's memory block. If you need to perform this action with a value held in a later offset of the variable memory block, you'd first need to load that value with the '**lda variable, offset**' instruction and store that value elsewhere before proceeding.	Bitcode: **62**

Items of Note:	If a divide by zero condition occurs, then a '**Divide by Zero**' error may be displayed. The '**flag**' '**divideByZeroErrorEnabled**' controls this behaviour. If set to '**false**', a zero value will be returned instead.

GT	Get Time
Description:	Retrieves the current Unix time stamp in milliseconds, and calculates the difference between the start and end times. The main purpose is for the creation of inline timers.

Allowed Patterns / usage:		
gt word	Based on the word provided, either the current time or difference will be returned to the '**A**' register.	Bitcode: **45**

Items of Note:	The '**gt**' instruction is constrained to only accept specific worded constants, even if a numeric constant is provided with the correct value; a compile error will be displayed.

HLT	Halt
Description:	Halts the entire program and awaits a key press to resume.

Allowed Patterns / usage:		
hlt	Halts the program and intermittently polls the keyboard to check for a key press. Once a key press is detected, the program resumes as normal.	Bitcode: **44**

Items of Note:	When the '**hlt**' instruction is first called, it erases any value stored in the '**A**' register. If this value is important, you, as the programmer, need to store it elsewhere. As with the '**in**' instruction, any bytes waiting in the keyboard buffer are consumed and stored in the '**A**' register. If the awakening key code is required, it will already be in the keyboard buffer on resume; no additional '**in**' instruction is required. The '**hlt**' instruction can be thought of as a replacement for the '**in**' instruction but one with the added functionality of halting the program and reducing CPU load at the same time. It is also important to note that while the system is halted, so is any other code, including '**timers**', the '**IRQ**' system, and '**yield phasing**'.

IDIV	**Integer Divide**
Description:	Used to divide the value held in the '**A**' register with the passed value but only return the whole number portion.

Allowed Patterns / usage:		
idiv constant	Divides the value held in the '**A**' register with the value of the passed constant.	Bitcode: **63**
idiv register	Divides the value held in the '**A**' register with the value held in the passed register.	Bitcode: **64**
idiv variable	Divides the value held in the '**A**' register with the first value held in the passed variable's memory block. If you need to perform this action with a value held in a later offset of the variable memory block, you'd first need to load that value with the '**lda variable, offset**' instruction and store that value elsewhere before proceeding.	Bitcode: **65**

Items of Note:	An integer division '**truncates**' the result of the division, returning only the whole number portions of the number. This is a better approach than using '**Floor**' or '**Round**' to provide the integer portion. If a divide by zero condition occurs, then a '**Divide by Zero**' error may be displayed. The '**flag**' '**divideByZeroErrorEnabled**' controls this behaviour. If set to '**false**', a zero value will be returned instead.

IN	**Reads the first waiting byte in the keyboard input stream or virtual port.**
Description:	Reads the first byte waiting in the keyboard input stream into the '**A**' register or controls operation of specified ports.

Allowed Patterns / usage:		
in	Reads the first byte waiting in the keyboard input stream and stores it in the '**A**' register. If no byte is waiting, '**0**' is returned.	Bitcode: **29**
in word	Read data from or controls the operation of a specified port in accordance with the function of the specified port. In the case of the '**keyBoardBuffer**' it reads the entire buffer into an appropriate '**db**' style variable array.	Bitcode: **30**
in word, word	Read data from or controls the operation of a specified port in accordance with the function of the specific port. This version of '**IN**' allows the inclusion of an addition '**word**' for more granular control.	Bitcode: **31**

INC	**Increment**
Description:	Increments the value in the '**A**' register by one.

Allowed Patterns / usage:		
inc	Increments the value in the '**A**' register by one.	Bitcode: **14**

JP	**Jump**
Description:	Adjusts the value of the program counter to the position of the specified label.

Allowed Patterns / usage:		
jp label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label.	Bitcode: **33**

Items of Note:	This is considered an '**unstructured**' jump and should not be used to jump in or out of a subroutine.

JG	Jump when Greater
Description:	Adjusts the value of the program counter to the position of the specified label.

Allowed Patterns / usage:		
jg label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register is greater than zero.	Bitcode: **36**

Items of Note:	This is considered an '**unstructured**' jump and should not be used to jump in or out of a subroutine.

JGZ	Jump when Greater or Zero
Description:	Adjusts the value of the program counter to the position of the specified label.

Allowed Patterns / usage:		
jgz label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register is greater than or equal to zero.	Bitcode: **37**

Items of Note:	This is considered an '**unstructured**' jump and should not be used to jump in or out of a subroutine.

JL	Jump when Less
Description:	Adjusts the value of the program counter to the position of the specified label.

Allowed Patterns / usage:		
jl label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register is less than zero.	Bitcode: **38**

Items of Note:	This is considered an '**unstructured**' jump and should not be used to jump in or out of a subroutine.

JLZ	Jump when Less or Zero
Description:	Adjusts the value of the program counter to the position of the specified label.

Allowed Patterns / usage:		
jlz label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register is less than or equal to zero.	Bitcode: **39**

Items of Note:	This is considered an '**unstructured**' jump and should not be used to jump in or out of a subroutine.

JSR	Jump to Subroutine
Description:	Adjusts the value of the program counter to the position of the specified label.

Allowed Patterns / usage:		
jsr label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label, at the same time the current program counter position is appended to the execution stack.	Bitcode: **40**

Items of Note:	This is considered a '**structured**' jump and is used for jumping to a subroutine terminated with the '**rts**' instruction.

JSRNZ	Jump to Subroutine when not Zero	
Description:	Adjusts the value of the program counter to the position of the specified label.	
Allowed Patterns / usage:		
jsrnz label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register is not equal to zero. At the same time, the current program counter position is appended to the execution stack.	Bitcode: **42**
Items of Note:	This is considered a '**structured**' jump and is used for jumping to a subroutine terminated with the '**rts**' instruction.	

JSRZ	Jump to Subroutine on Zero	
Description:	Adjusts the value of the program counter to the position of the specified label.	
Allowed Patterns / usage:		
jsrz label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register equals zero. At the same time, the current program counter position is appended to the execution stack.	Bitcode: **41**
Items of Note:	This is considered a '**structured**' jump and is used for jumping to a subroutine terminated with the '**rts**' instruction.	

JNZ	Jump when not Zero	
Description:	Adjusts the value of the program counter to the position of the specified label.	
Allowed Patterns / usage:		
jnz label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register is not zero.	Bitcode: **35**
Items of Note:	This is considered an '**unstructured**' jump and should not be used to jump in or out of a subroutine.	

JZ	Jump on Zero	
Description:	Adjusts the value of the program counter to the position of the specified label.	
Allowed Patterns / usage:		
jz label	Jumps to a position in code by adjusting the value of the program counter to that of the specified label if the value in the '**A**' register equals zero.	Bitcode: **34**
Items of Note:	This is considered an '**unstructured**' jump and should not be used to jump in or out of a subroutine.	

LDA	**Load Accumulator**
Description:	The ‘**lda**’ instruction is used to load a value into the ‘**A**’ register from a numeric constant, another register, a variable, or the memory address of a variable if the variable name is prefixed with the ‘**#**’ character. ‘**lda**’ is one of only two instructions capable of accepting an offset from the start of the variable’s memory block.

Allowed Patterns / usage:		
lda constant	Load data from a numeric constant.	Bitcode: **2**
lda register	Load data from another register.	Bitcode: **3**
lda address	Load the memory address of a variable using the ‘**#variableName**’ syntax.	Bitcode: **4**
lda variable	Load the first value of a variable’s memory block.	Bitcode: **5**
lda variable,constant	Load a value from a variable’s memory block using a numeric constant as an offset.	Bitcode: **6**
lda variable,register	Load a value from a variable’s memory block using the value stored in another register as an offset.	Bitcode: **7**
lda variable,variable	Load a value from a variable’s memory block using the first entry in another variable’s memory block as an offset.	Bitcode: **8**

Items of Note:	A system error will occur if the offset given is outside the bounds of the variable’s memory block.

LEN	**Get variable length**
Description:	Returns the number of elements stored within a variable’s memory block.

Allowed Patterns / usage:		
len variable	Returns the zero-based variable length by returning the ‘**maxOffset**’ of the variable from the symbols table. The ‘**A**’ register is populated with the result.	Bitcode: **46**

Items of Note:	This is useful for setting the length of a loop should there be a need to individually examine each value within the variable’s memory block.

LP	**Loop**
Description:	Adjusts the value of the program counter to the location of a label earlier in the code than the ‘**lp**’ instruction if the value in the ‘**C**’ register is greater than zero.

Allowed Patterns / usage:		
lp label	Jumps to a label in code by adjusting the value of the program counter if the value held in the ‘**C**’ register is greater than ‘**0**’. The value in the ‘**C**’ register is also decremented by one at the same time.	Bitcode: **32**

Items of Note:	The label being jumped to must be defined earlier in the code than the ‘**lp**’ instruction; otherwise, a compile error will be generated. As the loop counter ‘**C**’ is evaluated at the end of the loop, the ‘**0**’ value will also be counted within the loop range. So for a loop of ‘**3**’ you would place ‘2’ in the ‘**C**’ register. The upper and lower bounds of the range ‘**0–2**’ would all be considered valid. As all variables and data blocks are zero-based, this means you can use ‘**C**’ as the index for examining the memory block.

MOD	**Modulus Division**
Description:	Used to perform a modulus division on the value in the '**A**' register.

Allowed Patterns / usage:		
mod constant	Provides the remainder after dividing the passed constant into the value held in the 'A' register.	Bitcode: **73**
mod register	Provides the remainder after dividing the value held in the named register into the value held in the 'A' register.	Bitcode: **74**
mod variable	Provides the remainder after dividing the value held in the first byte of the named variable into the value held in the 'A' register. If you need to perform this action with a value held in a later offset of the variable memory block, you'd first need to load that value with the '**lda variable, offset**' instruction and store that value elsewhere before proceeding.	Bitcode: **75**

Items of Note:	If a modulus divide by zero condition occurs, then a '**Modulus divide-by-zero error.**' error may be displayed. The '**flag**' '**divideByZeroErrorEnabled**' controls this behaviour. If set to '**false**', a zero value will be returned instead.

MUL	**Multiply**
Description:	Used to multiply the value held in the '**A**' register with the passed value.

Allowed Patterns / usage:		
mul constant	Multiplies the value held in the 'A' register with the value of the passed constant.	Bitcode: **57**
mul register	Multiplies the value held in the '**A**' register with the value held in the passed register.	Bitcode: **58**
mul variable	Multiplies the value held in the '**A**' register with the first value held in the passed variable's memory block. If you need to perform this action with a value held in a later offset of the variable memory block, you'd first need to load that value with the '**lda variable, offset**' instruction and store that value elsewhere before proceeding.	Bitcode: **59**

NEG	**Negative**
Description:	Replaces the value held in the '**A**' register with the negative of that value.

Allowed Patterns / usage:		
neg	Replaces the value held in the '**A**' register with the negative of that value.	Bitcode: **66**

NOP	**No Operation**
Description:	As the title suggests, a '**nop**' is essentially an empty CPU cycle where nothing occurs, though that might at first seem like a useless instruction. A '**nop**' can be inserted to add additional CPU cycles should more accurate timing be required. The execution of a '**nop**' only takes a fraction of a millisecond, so they can be useful in this regard.

Allowed Patterns / usage:		
nop	Performs an empty CPU cycle.	Bitcode: **0**

Items of Note:	During compile, labels are converted to a '**nop**' so that their location in memory is preserved. Jumping to a label, though, does not cause the '**nop**' to be executed as the '**program counter**' is incremented immediately after the jump has occurred, moving it to the next instruction after the '**nop**'.

OUT	**Writes a value to console or the specified virtual port.**
Description:	Writes the value currently held in the '**A**' register to the console or '***virtual port***'.

Allowed Patterns / usage:		
out	Writes the value in the '**A**' register to the console as an ASCII glyph if the value is an integer no greater than '**127**'. If greater than '**127**', the operation is handed to the '**out word**' instruction to print the value inline with configure numeric print options.	Bitcode: **25**
out variable	Writes the quoted string variable's byte memory block to the console as ASCII glyphs in a single instruction, providing functionality for quickly printing textual values. If the specified variable is a '**file**' type and it has been opened in read mode. You can also quickly write the file contents to the screen.	Bitcode: **26**
out word	Writes the value to the '**virtual port**' provided as the parameter. Virtual ports are used for communicating with the '**OS**' or '**hardware**'.	Bitcode: **27**
out word, word	Writes the value to the '**virtual port**' with a specific worded parameter.	Bitcode: **28**

Items of Note:	The words are is in essence a switch to indicate the functionality required. When writing '**floating-point**' numbers to the console, the number of decimal places can be controlled with the '**floatingPointPrecision**' value pair, which can be adjusted with the '**sys**' instruction.

POP	**Stack POP operation**
Description:	Retrieves the last value placed onto the stack and stores it in the specified destination. At the same time, the last value on the stack is removed, and the stack length is decreased by one.

Allowed Patterns / usage:		
pop register	Retrieves the last value on the stack and stores it in the specified register.	Bitcode: **19**
pop variable	Retrieves the last value on the stack and stores it in the first memory address of a variable's memory block.	Bitcode: **20**

Items of Note:	If you wished to place the value into another location within the variable's memory block, then you would first need to '**pop**' the value into the '**A**' register, then use the '**sta**' instruction with an offset to store the value at the exact position in the memory block of the destination variable. With all '**push**' and '**pop**' operations, be aware that the stack works with a last-on, first-off configuration. You need to take this into consideration when using '**pop**' to restore multiple values. They need to be restored in the reverse order to that which was used to place the values on the stack.

POPA	**Stack POPA operation**
Description:	Restores the values of all system registers previously placed on the stack with the '**pusha**' or with multiple '**push**' instructions, though in best practice it is wise not to mix the two.

Allowed Patterns / usage:		
popa	Restore the values of all system register previously stored on the stack.	Bitcode: **70**

Items of Note:	Before proceeding, a check for a size mismatch of the stack will be performed. If there are insufficient values on the stack, then a runtime error will occur.

PUSH	**Stack Push operation**
Description:	Place the value of the specified constant, register or variable onto the stack.

Allowed Patterns / usage:		
push constant	Places a numeric constant on the stack.	Bitcode: **16**
push register	Places the value held in the specified register on the stack.	Bitcode: **17**
push variable	Places the first byte of a variable's memory block onto the stack.	Bitcode: **18**

Items of Note:	If you wished to place a later byte offset from the start of the variable's memory block, then you would need to first load that value into the '**A**' register using the required offset, then '**push**' the content of the '**A**' register onto the stack.

PUSHA	**Stack Pusha operation**
Description:	Places the values of all system registers onto the stack.

Allowed Patterns / usage:		
pusha	Places the current values of all system registers onto the stack.	Bitcode: **69**

Items of Note:	Before proceeding, a check of the free space on the stack will be performed. If there is insufficient stack space available to store all system registers, then a runtime error will occur.

RND	**Random**
Description:	Replaces the value held in the '**A**' register with a random number.

Allowed Patterns / usage:		
rnd	Replaces the value held in the '**A**' register with a random number between '**0**' and the current value.	Bitcode: **50**

Items of Note:	The '**rnd**' instruction requires a positive number larger than or equal to '**1**'. If that condition is not met, then the value in the '**A**' register will remain unchanged. If the value held in the '**A**' register is a floating-point type, then only the integer portion will be evaluated.

RTI	**Return From Interrupt**
Description:	Adjusts the value of the program counter to its prior position when the '**IRQ**' was triggered.

Allowed Patterns / usage:		
rti	Returns to a position in code by adjusting the value of the program counter, restoring the program counter's prior position stored on the '**execution stack**' when the '**IRQ**' was triggered. At the same time, the internal '**irqActive**' register is set to false.	Bitcode: **71**

Items of Note:	Unlike other '**structured**' or '**unstructured**' jump instructions or even the '**rts**' instruction, '**rti**' restores the program counter to the exact position it was at the time the '**IRQ**' was triggered, not the line after as in all other cases.

RTS	**Return From Subroutine**
Description:	Adjusts the value of the program counter to that of the subroutine call position.

Allowed Patterns / usage:		
rts	Returns to a position in code by adjusting the value of the program counter, restoring the program counter's prior position stored on the '**execution stack**' by the last subroutine call instruction.	Bitcode: **43**

Items of Note:	The '**rts**' instruction is considered a vital part of 'structured' programming and is used as the termination line of all subroutines, excluding an '**IRQ**' which is discussed in more detail later in this manual.

STA	Store Accumulator
Description:	The ‘**sta**’ instruction is used to store or copy the current value held within the ‘**A**’ register into a variety of different locations. It is one of only two instructions capable of writing data into a variable using an offset.

Allowed Patterns / usage:		
sta register	Stores the value in the ‘**A**’ register into a different register.	Bitcode: **9**
sta variable	Stores the value in the '**A**' register into the first byte of a variable’s memory block.	Bitcode: **10**
sta variable, constant	Stores the value in the '**A**' register into a specific byte of a variable’s memory block, using a numeric constant as an offset.	Bitcode: **11**
sta variable, register	Stores the value in the '**A**' register into a specific byte of a variable’s memory block, using the value held in another register as an offset.	Bitcode: **12**
sta variable, variable	Stores the value in the '**A**' register into a specific byte of a variable’s memory block, using the first byte of another variable as an offset.	Bitcode: **13**

Items of Note:	A system error will occur if the offset given is outside the bounds of the variable’s memory block.

STB	Set True Value Byte
Description:	Sets the specified destination, either register or variable, as true, ie ‘**1**’.

Allowed Patterns / usage:		
stb register	Loads the specified register with the value of ‘**1**’.	Bitcode: **76**
stb variable	Loads the first byte of a variable’s memory block with the value of ‘**1**’.	Bitcode: **77**

Items of Note:	This instruction is provided as a quick way of setting a flag in your Flax code as true, ie ‘**1**’. For example, in a game, you might set an item type flag to ‘**1**’ to indicate it is in your inventory or has been destroyed.

SQRT	Square Root
Description:	Replaces the value held in the ‘**A**’ register with the ‘Square Root’ of it’s value.

Allowed Patterns / usage:		
sqrt	Replaces the value held in the ‘**A**’ register with the ‘Square Root’ of it’s value.	Bitcode: **67**

SUB	Subtract
Description:	Used to subtract the passed value from the value held in the ‘**A**’ register.

Allowed Patterns / usage:		
sub constant	Subtracts the value of the passed constant from the value held in the ‘**A**’ register.	Bitcode: **54**
sub register	Subtracts the value held in the passed register from the value held in the ‘**A**’ register.	Bitcode: **55**
sub variable	Subtracts the first value in a variable’s memory block from the value held in the ‘**A**’ register. If you need to perform this action with a value held in a later offset of the memory block, you’d first need to load that value with the ‘**lda variable, offset**’ instruction and store that value elsewhere before proceeding.	Bitcode: **56**

SYS	**System Call**
Description:	Used for setting either system-level flags or key-value pairs.

Allowed Patterns / usage:		
sys word, word	Sets the specified system flag or key value pair to true or false or with the value held within the ‘A’ register. The new value needs to be loaded into the ‘**A**’ register before the ‘sys’ instruction is called.	Bitcode: **47**

Items of Note:	The ‘**sys**’ instruction is constrained to only accept specific worded constants. If a numeric constant is provided, even if of the correct value, a compile error will occur.

XCH	**Exchange**
Description:	Used to exchange the values within registers.

Allowed Patterns / usage:		
xch register	Exchanges (swaps) the value held in the ‘**A**’ register, with the value in the specified register.	Bitcode: **48**
xch register, register	Exchanges (swaps) the values held in the two specified registers.	Bitcode: **49**

IRQ SYSTEM

The Flax engine includes a software implementation of a retro-style IRQ or Interrupt system. Using IRQs, you can run multiple code streams simultaneously or use it as a background process to handle cleanup tasks not easily added to your main code stream.

The system is extremely simple to use. All that is required is an '**_irq:**' label/subroutine block terminated with an '**rti**' or return from interrupt instruction. Unless you wish to change any of the defaults using the '**sys**' instructions, little else is required. The compiler sets everything else up for you. For more information, consult the '**sys**' instruction reference or the constants table for flags and key-value pairs which control the IRQ's operation.

A point to note is that only one '**_irq:**' label and '**rti**' instruction are allowed, and it's good practice to back up and restore all the system registers when first entering or leaving the subroutine. The instructions '**pusha**' and '**popa**' have been implemented specifically with the IRQ system in mind.

The example code below is all that is required to set up an IRQ:

```
;Simple IRQ Template
_start:

mainLoop:

      jp mainLoop

;Catch line
      brk

_irq:
      pusha

      ;IRQ CODE

      popa
      rti
```

IRQ Example

During runtime, the IRQ is run within and after a configured number of '**Yield Phases.**' Yield Phasing is discussed more in that section, but in simple terms, it's when, after a set number of instructions have been executed, the runtime yields processor time back to the operating system, allowing other applications to update as well as communicating Flax's application status so that the runtime doesn't appear to be not responding in the Activity Monitor.

By default, the IRQ is set to trigger after twenty yield phases, and when not altered, a yield phase occurs after every thousand instructions.

These parameters are fully customisable using the '**sys**' instruction, allowing you to set either runtime flags or key-value pairs. You can even turn the IRQ system on or off at will by adjusting the '**irqEnabled**' flag as shown below.

Note, however, that IRQ-related flags or values cannot be modified during runtime if an IRQ subroutine has not been configured. Attempting to do so will generate a runtime error.

```
    lda false
    sys flag, irqEnabled
```

Disabling the IRQ

SUBROUTINES

As with everything else in Flax, the setup of a subroutine is quite simple. All that is required is a label terminated with an '**rts**' instruction. In truth, even the IRQ is a subroutine, though one which you can't directly jump to and which must be terminated with an '**rti**' instruction required by an IRQ.

```
mySubroutine:
      pusha

      //subroutine code

      popa
      rts
```

Subroutine setup

Subroutines are considered a part of '**structured programming**' and good program design. In Flax, they are called with the '**jsr**', '**jsrnz**', and '**jsrz**' instructions. You '***should never***' jump into or out of a subroutine using an '**unstructured jump**'. Doing so could result in stack overflow or underflow conditions, resulting in a system error.

It is also considered good practice to back up and restore any system registers you might be changing in that subroutine so that they are returned to their prior state on exit. The '**pusha**' and '**popa**' instructions make this simple. If you're only modifying one register, then you can singularly back up and restore that single register using the '**push**' and '**pop**' instructions, which require you to provide the register name as a parameter.

There is no restriction on subroutines calling other subroutines, as creating small subroutines of re-useable code is good practice and reduces code duplication. Be careful though that you don't unknowingly call any previously called subroutine from a later subroutine, as this will result in a circular call state and an execution stack overflow condition.

If you wish to pass parameters to another subroutine without using a predefined variable, you have two options. The simplest is to place the value you wish to pass into either register '**r0**' or '**r1',** and in fact, these registers were provided for exactly this purpose and are not backed up or restored with the '**pusha**' or '**popa**' instructions. The second and more tricky option is to place the value(s) you wish to pass onto the stack with the '**push**' instruction after you've restored any changes made to the system registers. If you've used the above example and have used '**pusha**' and '**popa**' to preserve the system registers, using '**r0**' or '**r1**' becomes a viable option and does not require the use of any permanent storage.

Note that when retrieving items from the stack, they must be restored in the reverse order to that in which they were placed. Also, you should never assume that the contents of '**r0**' and '**r1**' remain unchanged. They were provided explicitly for use as temporary storage when passing values to and from subroutines or other general tasks when permanent storage is not required. They are not backed up automatically in any way unless by yourself.

The only way to be one hundred percent certain that there has been no change to your parameter or return value is to define a variable as an implicit storage location.

If you wish to return a value from a subroutine when complete and don't wish to use any permanent storage registers, '**r0**' and '**r1**' are your best options.

If you don't wish to use registers, you can push the return value(s) onto the stack after restoring the system registers. In such a case, though, your return value(s) would need to be stored in '**r0**' or '**r1**' to avoid being overwritten, so such an approach would seem counterintuitive.

The best approach when choosing this path is to only back up and restore those registers you are changing singularly rather than using '**pusha**'. Use the '**push**' and '**pop**' instructions to do this, though remember to restore them in the reverse order in which they were placed. On the final line before the '**rts**' instruction, you would then place your return value on the stack. This can be tricky to get right the first time, as the process of restoring the registers at the end may result in your return value being overwritten. As stated previously, registers '**r0**' and '**r1**' are not affected by the '**pusha**' or '**popa**' instructions, so they are still your best solution; however, one of The Flax Project's core goals is to promote creative thinking and inventive solutions.

```
mySubroutine:
      pop r0      ;Get the passed value off the stack
      pusha       ;Backup the system registers

      //subroutine code

      popa        ;Restore the system registers
      push r0     ;Place your return value on the stack
      rts
```

Using the stack to retrieve and return values

REGISTERS

Flax has six signed 64-bit registers. Though their purposes are listed in the table below, when not being used for such, there is no limitation placed on user code storing values in any of these registers. It is recommended, however, that user code should store long-term values in defined variables, but if the need arises, the '**r0**' and '**r1**' registers have been set aside purely for temporary storage and are not affected by the '**pusha**' or '**popa**' instructions.

Register Name	Purpose
a	The '**A**' register is a system register and the one most frequently used, known as the accumulator. It is the resultant location for reading from the keyboard, files, timers, and others. All mathematical operations, comparisons, and branch instructions use this register as either an operand or as a conditional flag. The '**A**' register is also the only place where a value can be read to or written from.
b	The '**B**' register is a system register, most often used as a pointer when iterating through a variable's memory block.
c	The '**C**' register is a system register used as the counter within loop operations.
d	The '**D**' register is a general-purpose system register commonly used by background tasks.
r0	The '**R0**' register is a general-purpose register with no system purpose. It is an ideal location for storing temporary values without the need to define a variable or for passing values to or from a subroutine.
r1	The '**R1**' register is a general-purpose register with no system purpose. It is an ideal location for storing temporary values without the need to define a variable or for passing values to or from a subroutine.

TERMINAL CONTROL

From within Flax, the terminal is controlled using standard ANSI escape sequences. Those sequence IDs, names, purpose, and index are listed in the table below.

To use any of the predefined sequences or constants, you use the '**out terminal, constant**' syntax. For those sequences which require additional information such as '**cursorColumn**' and '**cursorRow**' you must load the required value into the '**A**' register, using '**lda**' before use.

Flax provides a basic subset of escape sequences, but there is nothing stopping you from creating your own sequences with a '**db**' byte, or quoted string, type variable, and just outputting it with the '**out variableName**' syntax.

```
    out terminal, brightBlue

    lda 10
    out terminal, cursorColumn
```

Using a terminal control sequence.

Constant	Value	Type	Usage
clearScreen	0	control	Clears the screen but does not home the cursor to the top left of screen.
clearLine	1	control	Clears the line the cursor is on.
clearHistory	2	control	Clears the scroll back history but not the screen.
setRowCount	3	Control	Sets the terminal row count to the value loaded in the 'A' register.
cursorScreenHome	4	control	Moves the cursor to the top left of the screen.
cursorLineHome	5	control	Moves the cursor to the start of the current line.
cursorUp	6	control	Moves the cursor up one row.
cursorDown	7	control	Moves the cursor down one row.
cursorRight	8	control	Moves the cursor right one column.
cursorLeft	9	control	Moves the cursor left one column.
cursorColumn	10	control	Moves the cursor to the specified column loaded into the '**A**' register.
cursorRow	11	control	Moves the cursor to the specified row loaded into the '**A**' register.
cursorHide	12	control	Hides the cursor.
cursorShow	13	control	Shows the cursor.
styleReset	14	style	Resets all text styles and colours for all subsequent characters.
styleBold	15	style	Sets the text style to bold for all subsequent characters.
styleItalic	16	style	Sets the text style to italic for all subsequent characters.
styleUnderLined	17	style	Sets the text style to underlined for all subsequent characters.
styleBlinkOn	18	style	Enables the text blinking style for all subsequent characters.
styleBlinkOff	19	style	Disables the text blinking style for all subsequent characters.
colorReset	20	colour	Resets both the foreground and background colour to terminal default.
colorResetFC	21	colour	Resets the foreground colour to terminal default.
colorResetBC	22	colour	Resets the background colour to terminal default.

Constant	Value	Type	Usage
dimBlack	23	colour	Sets then foreground colour for all subsequent characters to dim black.
dimRed	24	colour	Sets the foreground colour for all subsequent characters to dim red.
dimGreen	25	colour	Sets the foreground colour for all subsequent characters to dim green.
dimYellow	26	colour	Sets the foreground colour for all subsequent characters to dim yellow.
dimBlue	27	colour	Sets the foreground colour for all subsequent characters to dim blue.
dimMagenta	28	colour	Sets the foreground colour for all subsequent characters to dim magenta.
dimCyan	29	colour	Sets the foreground colour for all subsequent characters to dim cyan.
dimWhite	30	colour	Sets the foreground colour for all subsequent characters to dim white.
black	31	colour	Sets the foreground colour for all subsequent characters to black.
red	32	colour	Sets the foreground colour for all subsequent characters to red.
green	33	colour	Sets the foreground colour for all subsequent characters to green.
yellow	34	colour	Sets the foreground colour for all subsequent characters to yellow.
blue	35	colour	Sets the foreground colour for all subsequent characters to blue.
magenta	36	colour	Sets the foreground colour for all subsequent characters to magenta.
cyan	37	colour	Sets the foreground colour for all subsequent characters to cyan.
white	38	colour	Sets the foreground colour for all subsequent characters to white.
brightBlack	39	colour	Sets the foreground colour for all subsequent characters to bright black.
brightRed	40	colour	Sets the foreground colour for all subsequent characters to bright red.
brightGreen	41	colour	Sets the foreground colour for all subsequent characters to bright green.
brightYellow	42	colour	Sets the foreground colour for all subsequent characters to bright yellow.
brightBlue	43	colour	Sets the foreground colour for all subsequent characters to bright blue.
brightMagenta	44	colour	Sets the foreground colour for all subsequent characters to bright magenta.
brightCyan	45	colour	Sets the foreground colour for all subsequent characters to bright cyan.
brightWhite	46	colour	Sets the foreground colour for all subsequent characters to bright white.
bgBlack	47	colour	Sets the background colour for all subsequent characters to black.
bgRed	48	colour	Sets the background colour for all subsequent characters to red.
bgGreen	49	colour	Sets the background colour for all subsequent characters to green.
bgYellow	50	colour	Sets the background colour for all subsequent characters to yellow.
bgBlue	51	colour	Sets the background colour for all subsequent characters to blue.
bgMagenta	52	colour	Sets the background colour for all subsequent characters to magenta.
bgCyan	53	colour	Sets the background colour for all subsequent characters to cyan.
bgWhite	54	colour	Sets the background colour for all subsequent characters to white.
bgBrightBlack	55	colour	Sets the background colour for all subsequent characters to white.
bgBrightRed	56	colour	Sets the background colour for all subsequent characters to bright black.
bgBrightGreen	57	colour	Sets the background colour for all subsequent characters to bright red.
bgBrightYellow	58	colour	Sets the background colour for all subsequent characters to bright yellow.
bgBrightBlue	59	colour	Sets the background colour for all subsequent characters to bright blue.
bgBrightMagenta	60	colour	Sets the background colour for all subsequent characters to bright magenta.
bgBrightCyan	61	colour	Sets the background colour for all subsequent characters to bright cyan.
bgBrightWhite	62	colour	Sets the background colour for all subsequent characters to bright white.

Constants in this table can only be used with the '**out terminal, word**' instruction syntax.

** The operation of the listed control sequences depends entirely on the configuration or capabilities of your terminal; none, some, or all may function as expected or not at all.*

** On macOS you may need to disable '**Dynamic Foreground Colours**' for the colour codes to be displayed with the correct Hue and Saturation.*

TIMERS

Flax supports an unlimited number of software or inline timers, with each being able to control its own independent code stream, giving Flax the appearance of running multiple tasks at the same time. Though not technically true, the speed of Flax's engine is such that running separate code streams in this manner is a viable option.

As with most things in Flax, configuration and implementing a timer is a simple process. The '**gt**' *(get time)* instruction, along with its parameters of '**timerTimeNow**' or '**timerTimeSince',** paired with a comparison operator, covers most of what's required.

```
_start:
      gt timerTimeNow
      sta timer

mainLoop:
      lda timer
      gt timerTimeSince
      cmp 1000 ;one seconds
      jg notExpired
      jsr timerExpired
notExpired:

      jp mainLoop

catch:
      brk

timerExpired:
      ;Timer Code

      gt timerTimeNow
      sta timer
      rts

dd timer, 0
```

Timer Example

The parameters passed to the '**gt**' instruction are used to first get the start time, then secondly calculate the difference between them. Paired with the comparison operator '**cmp',** it's easy to configure as many repeating or single-shot timers as you require.

Note: For granularity, timer times are returned in milliseconds, so 1000 would be one second, and 500 half a second.

All timers need to be checked within a frequently running code stream, and for repeating timers, the timer needs to be reset whenever it expires.

VARIABLES

Flax has a reasonably mature variable system for an assembly-like language. There are a number of different types, including unsigned bytes, signed integers, floating-points, as well as memory blocks. Excluding the byte (**db**), all are 64-bit signed values, allowing for positive or negative numbers.

There is '**no**' implicit string variable type. Even if a quoted string is provided as the default to a byte (**db**) type, it will be converted into a byte-list containing the ASCII values of the characters within the string terminated in memory with a '**0**' or null value.

Casting or changing a value from one type to another is a simple process in which you assign the value to an alternate variable of the required type. There is no implicit cast operator, and everything is handled automatically by the system.

Note, however, that assigning a value too large for a smaller unsigned type causes wraparound, also called integer overflow. The value wraps around when it reaches the type's limit. In other words, the result is the remainder after the original value is divided by the number of values the type can hold.

Another consideration is when assigning a double (**dd**) value to an integer (**dw**) or byte (**db**). Any floating-point values of the original value will be lost, and the result will be the whole number portion.

Although both types use 64 bits, integers (**dw**) and floating-point numbers (**dd**) use those bits differently. Integers are better for storing exact whole numbers, while floating-point numbers can represent a much wider range of values but may lose accuracy when storing very large whole numbers.

How each type is defined is similar, but there are differences to be aware of. Variables are defined with the variable type first, followed by the name and then the default value(s). All are mandatory and cannot be excluded. There is no way to define a variable and have the system provide a default value.

Other considerations are that variable memory is assigned during compilation, and it is not possible during runtime to alter the size of a variable's assigned memory block. It is also not possible to use a constant such as '**pi**' as the default. Variable definition(s) have to be performed well before the constant replacement process, so you are required to enter the value directly. In this case, however, such a restraint is not a drawback as you're able to

view your code and know at a glance what value the variable contains, which is not immediately clear with a constant.

The symbols table contains a lookup list with a '**maxOffset**' key for every variable defined. This key can be later used to return the number of elements within a variable's memory block. The '**len**' instruction is used for this purpose. You can learn more about its operation in the instructions reference section of this manual.

Declarations

Byte (db)

The byte variable type is an unsigned positive non-floating-point value in the range '**0 - 255**' inclusive. If you have a requirement for negative numbers in this range, it is not possible with this type; you have no option other than using the (**dw**) or signed integer type instead.

Although declarations of all types are similar, the byte is the only type capable of accepting a quoted string as its default value.

```
;A single byte value
db variableName, 0

;A byte list
db variableName, 0, 1, 2, 3, 4 ,5

;A quoted string
db variableName, "Hello, Flax World!”

;A byte type memory block
db variableName, #10
```

Byte Type Declarations.

If you wish to have quotes contained within your string, they are required to be '*escaped*' with a '\' character. The same is true when including control characters such as tabs and line feeds.

```
;A quoted string with control characters
db variableName, "Hello,\t \"Flax\" World!\n"
```

Byte Type String Formatting example.

Word / Integer (dw)

The word or integer variable type is a signed 64-bit non-floating-point value, allowing for positive and negative values within the range a 64-bit value can contain.

```
;A single word value
dw variableName, 0

;A word list
dw variableName, 0, 1, 2, -3, 4 ,5

;A word memory block
dw variableName, #10
```

Word (Integer) Type Declarations.

Double (dd)

The double variable type is a signed 64-bit floating-point value, allowing for positive and negative values within the range a 64-bit floating-point value can contain.

```
;A single double value
dd variableName, 3.14159

;A double list
dd variableName, 0, 1.5, 2, -3, 4 ,57.6

;A double memory block
dd variableName, #10
```

Double (Floating-Point) Type Declarations.

Files

Within Flax, files are treated like any other variable type. Their declaration is almost identical to that of a '**db**' quoted string. Note: only files which are in the current directory or a subdirectory of the same will be accepted.

This limitation has been added to enforce strict file system security rules.

```
;A file definition
file variableName, "test.txt"
```

File Type Declarations.

Memory Blocks

Any variable type can also be defined as a memory block using the '**#value**' syntax. When defining a memory block, the size must be a positive integer of '**1**' or greater.

```
;Byte type memory block.
db variableName, #10

;Word or integer type memory block.
dw variableName, #10

;Float or double type memory block.
dd variableName, #10
```

Memory-Block Declarations.

This method of declaration is a far quicker way of creating a large container of values without needing to list each individually. The memory block also differs from other variables in that each element is initialised with the value of '**0**'.

Variable Lists

The list type declaration of variables and even the single value assigned to a memory-block can be though of as and array type definition in other languages.

Using Offsets

Regardless of variable type or the number of elements contained, the symbols table for the compiled program contains a '**maxOffset**' key for each variable. This value, which is retrievable with the '**len**' instruction, is provided as a simple way of getting the total number of elements a variable's memory block contains.

This value, when used in conjunction with a simple pointer, provides an easy method to iterate through the elements of a variable's memory block, examining each in turn.

It is important to note, however, that only the '**lda**' and '**sta**' instructions are capable of accepting an offset. You should also be aware that trying to read or write to a value outside the bounds of the variable will result in a system error.

A good example of using an offset with a pointer is the first and second '**Hello, Flax World!**' examples close to the start of this manual.

Note! When using an offset when reading from or writing to a file, the action alters the file read or write pointer; for more information, consult the '**FileIO**' section of this manual.

VIRTUAL PORTS

Virtual ports are really only ports in name more than function. The parameters provided on the '**in word**' or '**out word**' instructions are in reality just switches which enable the functionality to read or write to the correct '**OS**' *operating system* areas.

As indicated, only the '**in**' and '**out**' instructions are allowed to read or write to ports.

The ports or '**words**' used by the instructions are listed below.

Port Name	Value	Type	Used By	Usage
number	0	switch	out	Forces the '**out**' instruction to output the value within the '**A**' register as a number regardless of value.
terminal	1	switch	out	Instructs the '**out**' instruction to output a terminal control sequence ID, either generated programmatically and stored in the '**A**' register with '**out terminal**', or a known word as the second parameter, e.g. '**out terminal brightBlue**'. If an invalid terminal control sequence is provided, then a system error will occur.
keyBoardBuffer	2	switch	in	Reads the entire '**keyBoardBuffer**' into a user-defined 64-byte '**db**' type byte array. The '**A**' register needs to be loaded with the variable address, using the '**lda #variableName**' syntax before calling '**in keyBoardBuffer**'. If an invalid variable type or size is provided then a system error will occur.
fileIO	3	Switch	in / out	Port used for controlling all fileIO operations within the system. Consult the '**FileIO**' section in this manual for in-depth information regarding its use.

Constants in this table can only be used with the instruction listed.

YIELD PHASING

Yield Phasing is the process employed by the Flax runtime to offload CPU time back to the operating system and other applications. It also allows the system to update the status of the Flax process so that it doesn't appear as not responding within the Activity Monitor.

The time and frequency of yield phases are controlled with the '**Key-Value Pair**' '**sysPerformance**' which controls how many assembly language instructions are executed before a yield occurs.

The default of '**1000**' instructions should provide ample performance for most situations, though you have the freedom to adjust this value between '**5** and **50,000**' inclusive; however, higher settings may increase performance but they also incur the penalty of increased CPU load.

During a yield phase, there are three other things which may also occur: the release of CPU time with a system-level '**thread.yield**', an '**autoPause**' may run if the appropriate '**flag**' has been set, and lastly, an IRQ may trigger if the IRQ system has been enabled and the configured number of yield phases has been reached since the last IRQ operation.

System Yield

A system yield is the simplest and default action that will occur in a yield phase if no other options have been set. This is an OS-level system call to release CPU time from the Flax process to any other waiting processes.

This has very little impact on lowering CPU usage, but it does allow the Flax process status to be updated by the system so that it doesn't appear as though it isn't responding.

Auto Pause

If Auto Pause is configured by setting the '**autoPause**' flag to true, it replaces the above system yield operation in favour of sleeping the runtime thread for a configured number of milliseconds.

The auto-pause time, if you wish to change it from default, can be set with the '**Key-Value Pair**' '**autoPauseTime**'.

Setting an auto-pause in favour of a standard system yield has a huge impact on CPU usage, often dropping it to almost zero. As with the system yield, during an auto-pause, the status of the Flax process is communicated with the system.

Enabling an '**auto-pause**' is the recommended option in most cases, especially if your program is mostly idle waiting for user interaction, so you might be wondering why it isn't set as default.

Learning the system and understanding its quirks and intricacies is again our core focus. If we were to enable auto-pause by default, you would probably never know or go looking for a solution to lower your program's CPU usage.

IRQ activation

The last thing which might occur during a yield phase is the triggering of an IRQ. IRQs, if configured, are triggered after a set number of yield phases and function much like a background thread to run tasks not easily implemented into your main code stream.

Consult the IRQ section for more detail on their setup, but to get started, their frequency of triggering can be easily adjusted with the '**irqPerformance**' '**Key-Value Pair**'.



CHANGE LOG

This section lists any and all changes to publicly available builds, alpha, beta, or release builds alike.

FLAX CHANGE LOG

Version	Changes / Notes
Beta 1.5	1. Corrections to compiler profiling rules to better calculate the difference between subroutine jumps and return statements. 2. Correction to code profiler rule which was negating the above.

Version	Changes / Notes
Beta 1.4	1. Improvements to the full compiler snapshot to make it more readable as well as taking less screen space. 2. Bug fixes. 3. Stability Improvements.

Version	Changes / Notes
Beta 1.3	1. Further Improvements to the encryption system. 2. Better checking and reporting of flax library issues.

Version	Changes / Notes
Beta 1.2	1. Implemented a change to the file encryption system to better identify encrypted files. It is important that you do not encrypt your source file unless you intend to distribute it. It is not possible to edit the source code of an encrypted file.

Version	Changes / Notes
Beta 1.1	1. Implemented a fix to allow compiler and runtime snapshots to be displayed at any time, not only after an error during compilation or execution.

Version	Changes / Notes
Beta 1.0	Initial Release